VNAP-Secure Documentation

Design intention, setup, operation, reference, patch set and results

Compiled from the Markdown documentation in the repository
sources last changed 2026-10-17 · digest aa0ea90de395

Contents

Introduction: what VNAP-Secure is for 4

The question 4

Design intentions 4

How the parts fit 5

Limits by design 5

How to read this document 5

Overview 6

VNAP-Secure 7

Architecture 11

Installation and build 14

Installation 15

Build 17

Certificates 19

Test certificates 21

How it works 22

Security and chain verification 23

Pseudonym change 25

Certificate refill (per-run PKI) 28

Vehicle movement 31

Eavesdropper and scoring 35

Operations 40

vnapctl 41

Service deployment 46

Web UI 50

Backup and restore 53

Monitoring 55

Troubleshooting 57

Reference 59

Scenario format 60

Configuration 64

Container environment 66

Control channels 68

API 70
Scenarios 73

Patch set 74

Patch documentation 75
Patches: build and container 78
Patches: certificate verification 79
Patches: thread safety 81
Patches: pseudonym and ID change 83
Patches: certificate refill 85
Patches: mobility and CAM 86

Results 87

Test summaries 88

Appendix 90

Changelog 91
ASN.1 modules 98

Introduction: what VNAP-Secure is for

written for this document

The question

Connected vehicles broadcast Cooperative Awareness Messages (CAMs) several times a second. Each one carries the vehicle's position, speed and heading, and each is signed with a certificate (ETSI TS 103 097, IEEE 1609.2) so that receivers can trust it. The signature that makes a message trustworthy also makes it traceable: a certificate, a MAC address or a station ID that stays the same lets anyone with a receiver follow a vehicle from street to street.

The standards answer this with pseudonyms. A vehicle holds a pool of short-lived authorization tickets and changes to a new one from time to time, together with every other identifier it sends. Whether that protects anybody depends on details the standards leave open: whether all identifiers really change at once, whether anything else carries over (the rhythm of the messages, the position), whether the vehicle falls silent for a moment, where it changes, and how many other vehicles could plausibly be the one that reappears.

VNAP-Secure exists to answer that question by measurement rather than argument: **how well do pseudonym changes protect vehicles against tracking, and what do they cost?**

Design intentions

The project is built around a few decisions, each of which shapes the chapters that follow.

- **The real protocol stack, not a model of it.** Stations run Vanetza-NAP's `socktap`, an implementation of the ETSI ITS-G5 stack, with C-ITS security switched on. Where upstream does not do what a deployment must (verify the whole certificate chain, change all identifiers together under the ETSI ID change notification, hold and refill a pseudonym pool, survive concurrent load), the project adds a patch set rather than simulating the behaviour. The frames on the wire are standard frames that any receiver, or Wireshark, can decode.
- **An honest attacker.** The eavesdropper is a passive listener that sees only what a roadside receiver would see: the frames on the message network. It holds no keys, is not on the control network, and never sees ground truth. Its linking techniques are documented one by one, each with the countermeasure that defeats it.
- **Ground truth kept apart from the attacker's view.** Which certificate each vehicle really used comes from the stations' own logs and the PKI's issue log. Every link the eavesdropper makes is scored correct or wrong against it, so a privacy result is a count, not an impression.
- **A message plane and a control plane.** What a real vehicle would know internally or receive over a backend link (its GNSS position, the decision to change pseudonym, the connection to its certificate authority) travels on a separate control network that the attacker cannot reach. This keeps the simulation's own machinery from leaking into what is measured.
- **Certificate handling close to a deployment.** Each run can have its own PKI. Vehicles start with a batch of butterfly-expanded authorization tickets and request new batches as they run low; by default they derive the private keys themselves, so the PKI never holds them, and no certificate is used twice. The time to refresh certificates is measured, because a vehicle without an unused pseudonym cannot change.
- **Measured countermeasures.** Each defence was added because a measurement showed an attack that worked: full ID change against shared identifiers, silent periods against position continuity, mix zones against a patient attacker, a random CAM phase against timing, random turns against fixed traffic patterns. The test summaries record each step with its numbers.

- **Reproducible runs.** A run is defined by a scenario file, a seed and image tags, and records them together with the C-ITS-PKI commit it used. The eavesdropper's frames are kept, so its linking can be replayed with other settings without running the simulation again.
- **Usable without a shell.** A multi-user service puts accounts, quotas and isolation in front of Docker. Users author scenarios from templates, in a builder or as text; watch runs live; and receive a summary report that states what was measured and, by fixed rules, which run to try next.

How the parts fit

Part	Role	Chapters
Patch set	makes `socktap` behave like a deployed station: chain verification, ID change, pseudonym pools, refill, controllable position, thread safety	How it works; Patch set
Simulation harness (`vnapctl`, `vnapsim`)	starts stations, control broker, per-run PKI, mobility client and eavesdropper from a scenario file; status, events, checks and scoring	Operations; Reference
Service	the same for many users, through an HTTP API and a web UI, with results and reports kept per run	Operations; Reference
Results	test summaries of the privacy scenarios, scored against ground truth	Results

Limits by design

VNAP-Secure runs on a single host. Stations exchange frames over a Docker bridge, which loses nothing and has no range, so there is no radio model. Traffic is a handful of vehicles on routes and crossings. Results are therefore comparisons between countermeasures under stated conditions, not predictions for a road network. Certificate revocation, misbehaviour reporting and encrypted delivery of certificate batches are outside the scope, as stated in the product requirements.

The design proposals in `Specs/` describe where the project is meant to go next: adversarial agents that evolve attacks and defences against each other, and measurements of the computational and safety cost of pseudonym changes, weighed against privacy in a joint experiment.

How to read this document

This document collects the Markdown documentation of the repository as it stood when it was generated. The parts follow the README's documentation table:

- **Overview:** the README and the architecture, with networks and trust boundaries.
- **Installation and build:** the VM, the image build and the certificate sets.
- **How it works:** security and chain verification, pseudonym change, certificate refill, vehicle movement, and the eavesdropper with its scoring.
- **Operations:** `vnapctl`, deploying and using the service, backups, monitoring and troubleshooting.
- **Reference:** the scenario format, configuration, container environment, control channels and the API.
- **Patch set:** each group of changes to Vanetza-NAP, with its reason and status.
- **Results:** the test summaries.
- **Appendix:** the changelog and source notes.

Links between documents are links within this PDF. Links to source files point to the repository on GitHub. The Markdown files remain the source; this PDF is a snapshot of them.

Overview

VNAP-Secure

README.md

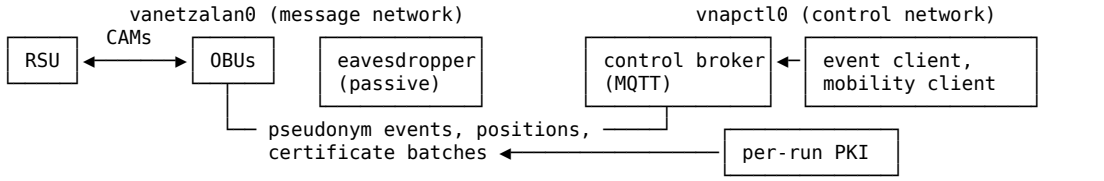
with an introduction to the design intention of VNAP-Secure.

VNAP-Secure runs Vanetza-NAP (`release2-main`) V2X stations with C-ITS security enabled (ETSI TS 103 097: `certs-v2` for V1.2.1 certificates, `certs-v3` for V1.3.1 / IEEE 1609.2 certificates), and studies how well pseudonym changes protect vehicles against tracking. It provides:

- a **patch set** for Vanetza-NAP's `socktap`:
  - full certificate-chain verification;
  - thread-safety fixes;
  - pseudonym pools with event-driven changes and ETSI ID change notification;
  - certificate refill from a PKI with on-board butterfly key derivation;
  - a runtime-controllable position;
- a **simulation harness** (`vnapctl`) that starts RSUs and OBUs from scenario files, together with a pseudonym control channel, a per-run PKI, a mobility client that drives the vehicles, and a passive eavesdropper that tries to track them;
- a **multi-user service** (HTTP API and web UI) to author, run, observe and evaluate scenarios without the command line.

This is not the Vanetza-NAP source tree: the build fetches it and lays the patch set over it. Certificates come from the companion C-ITS-PKI project, included as a git submodule.

Components



`vnapctl (sim/)` starts and inspects a run; the service (`service/`) does the same for many users

Component	What it does	Documentation
Stations (`vnap:latest`)	patched `socktap`: signed CAMs, chain verification, pseudonym changes, refill	security, pseudonym change
Per-run PKI (`vnap-pki`)	the run's CA hierarchy; issues butterfly authorization tickets in batches	certificate refill
Mobility client (`vnap-mobility`)	drives vehicles along routes or random-turn crossings; mix zones	vehicle movement
Eavesdropper (`vnap-eavesdropper`)	passive tracking attacker; scored against ground truth	eavesdropper and scoring
`vnapctl` / `vnapsim`	scenario validation, start/stop, status, events, checks	vnapctl
Service	accounts, runs, live view, control, results, backups; web UI	deployment, web UI

The whole picture, with networks and trust boundaries, is in docs/architecture.md.

Repository layout

Path	Content
`patches/vanetza-nap/`	the patch set: `modified/` files, their `upstream/` originals, generated `diffs/`, the upstream `BASE` commit
`sim/`	`vnapctl` and the `vnapsim` package, `scenarios/` (and `templates/`), helper images in `images/`, the user policy, tests

Path	Content
service/	the HTTP API (vnapapi/), web UI (ui/), deployment files (deploy/), configuration, tests
certs/	committed **test** certificate sets: certify/ (v2) and c-its-pki/ (v3)
external/C-ITS-PKI/	git submodule: the PKI library used by the per-run PKI
scripts/	build/ (image, diffs), vm/ (development VM helpers), certs/ (certify tools), docs-check.py
docs/	installation, build, operations, reference, functional descriptions, patch documentation
results/	test summaries of the privacy scenarios
Specs/	product requirements, privacy paper, the repository plan

Requirements

- **An Ubuntu VM** (24.04; development used VMware on a Mac). Vanetza-NAP has file names that differ only by case, so it cannot be built on a case-insensitive file system.
- **Docker, git, make, python3 ≥ 3.11** in the VM.
- **About 10 GB of free disk** for the first image build.
- **Access to GitHub:** the C-ITS-PKI submodule and the upstream vanetza-nap sources.

Installation

```
INSTALL_DIR=<desired installation directory>
git clone --recurse-submodules git@github.com:jodyhuntatx/vnap-secure.git ${INSTALL_DIR}/vnap-secure
export VNAP_HOME=${INSTALL_DIR}/vnap-secure
cd ${VNAP_HOME}
git submodule update --init      # in a clone made without --recurse-submodules
```

Shared folders from the host, snap-confined Docker, git on /mnt/hgfs and disk sizing are covered in docs/installation.md.

Build

```
cd ${VNAP_HOME}
sudo apt install -y make docker.io     # install make and docker
sudo usermod -aG docker $USER          # add current user to docker group for non-sudo docker access
<logout/login to effect non-sudo docker access>
make image          # fetch vanetza-nap at patches/vanetza-nap/BASE into ~/vanetza-nap, apply the
    patches, build vnap:latest, takes ~20+ minutes
docker run --rm --entrypoint /usr/local/bin/socktap vnap:latest --help | grep pseudonym-control   #
    patched?
```

- **Time:** the first build takes about 20 minutes; later builds reuse cached layers.
- **Helper images** (PKI, mobility client, eavesdropper, control client) are built by vnapctl on demand.
- **Details:** VANETZA_NAP_DIR and IMAGE, building the unpatched sources (make origs) and releasing an image tag are in docs/build.md.

Run a simulation

```
cd ${VNAP_HOME}/sim
./vnapctl scenarios                     # the five foundational scenarios, their variants, controls
./vnapctl up trust                      # validate, start, wait until the stations exchange messages
./vnapctl status                        # stations, certificates, chain checks
./vnapctl check                         # observe 15 s: PASS/FAIL against the expectations
./vnapctl events --duration 10s         # received/sent messages, pseudonym and PKI events
./vnapctl down                          # remove containers, networks and the run's keys
```

The scenarios each isolate one phenomenon, and each has variants (<scenario>:<variant>):

- trust: certificate-chain verification (trust:badroot is the negative control).
- linkability: what a pseudonym change leaks when a vehicle is alone.

- `crossing`: four cars changing identity in a mix zone, watched by an eavesdropper.
- `convoy`: cars that change together but never separate.
- `refill`: the run's own PKI with certificate refill.

`--instance auto` runs a second, independent simulation next to the default one.

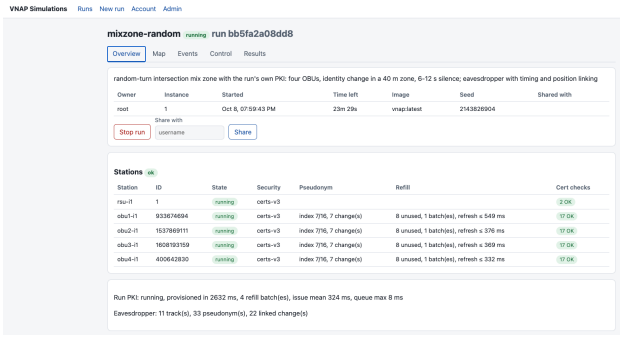
- **Overrides and instances:** docs/operations/vnapctl.md.
- **Scenario format:** docs/reference/scenario-format.md.

Run the service

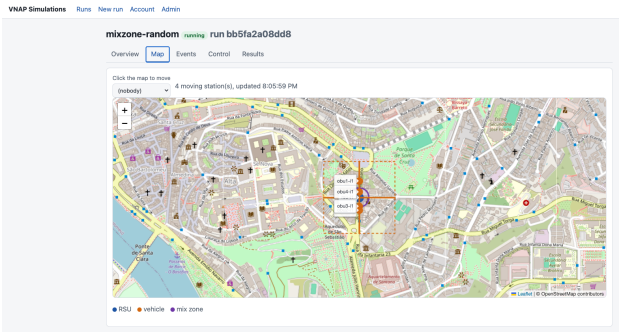
```
cd ${VNAP_HOME}/service
./0-install-venv.sh                          # install virtual env support and initialize
./create-admin.sh                            # first admin (password asked, or
    VNAP_NEW_PASSWORD)
./run.sh                                     # 0.0.0.0:8080, web UI at /ui/, API docs at
    /api/docs
```

The service listens on all interfaces (`0.0.0.0`) by default. This is required when port 8080 is forwarded from the VM to the host (e.g. to open the UI at `http://localhost:8080` on a Mac). Set `VNAP_API_HOST=127.0.0.1` to listen on the VM only.

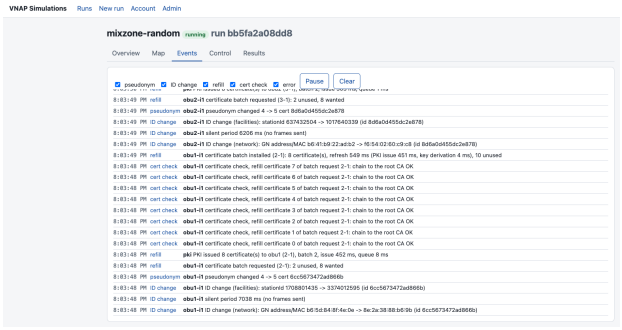
In production the service runs as a dedicated account under systemd, behind Caddy for TLS (`${VNAP_HOME}/service/deploy/`). See docs/operations/service-deployment.md, then docs/operations/web-ui.md for users.



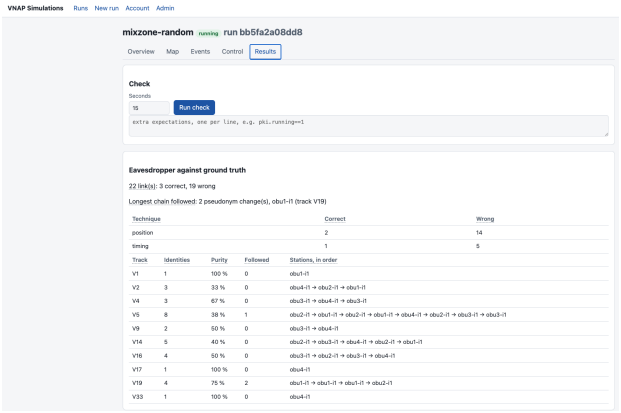
Overview



Map



Events



Results

Operate

- **Monitoring:** decoded messages over MQTT, frames with tcpdump/Wireshark, events and logs. See docs/operations/monitoring.md.
- **Backups and restore:** daily archives, the restore runbook, and custody of the TOTP key. See docs/operations/backup-restore.md.

- **Troubleshooting:** startup chain checks, offline message checks, known limitations. See docs/operations/troubleshooting.md.

Tests

```
cd ${VNAP_HOME}
make test        # vnapsim and service unit tests, web UI syntax, OpenAPI spec and patch diffs current,
    documentation links, documentation PDF current
```

None of these need a running simulation. Changes that affect running stations are checked with live scenario runs (`vnapctl up … && vnapctl check`), as described per component in the docs.

Documentation

Area	Documents
Setup	installation · build · architecture · certificates
Operations	vnapctl · service deployment · web UI · backup and restore · monitoring · troubleshooting
Reference	scenario format · configuration · container environment · control channels · API (OpenAPI)
How it works	security and chain verification · pseudonym change · certificate refill · vehicle movement · eavesdropper and scoring
Patch set	patch documentation
Results	test summaries
Specifications	`Specs/VNAP-Secure-Simulation-Service-PRD.pdf`, `Specs/VNAP-Secure-Repository-Reorganization-Plan.docx`

The general Vanetza-NAP documentation (configuration, MQTT/JSON formats, applications) is in the vanetza-nap tree (`docs/`) and at https://wiki.nap.av.it.pt/groups/nap/tutorials/vanetza-nap/; it does not cover security.

Versions

The current station image is `vnap:r2-p19` (also `vnap:latest`). Image tags, features by date and the history of this repository are in CHANGELOG.md.

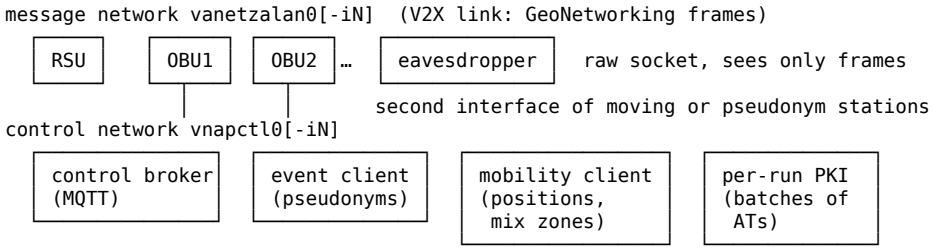
Moved sections. The sections of the earlier single README are now in `docs/`; the changelog maps each old section to its new place.

Architecture

docs/architecture.md

A simulation run

A run is a set of containers on two Docker networks of its own:



- **Stations** run the patched `socktap` (`vnap:latest`), each with its embedded MQTT broker for application messages (`vanetza/in|out|own/*`). Stations with pseudonyms, movement or refill also join the control network, on a separate MQTT connection.
- **Control network:** carries pseudonym change events and answers, position updates and certificate batch requests (control channels). RSUs and the eavesdropper are not on it.
- **Instances:** instance 0 uses `vanetzalan0` / `vnapctl0` (192.168.98.0/24, 192.168.99.0/24). Instance N uses `-iN` networks with 10.N.98.0/24 and 10.N.99.0/24 and container names `<name>-iN`, so runs never share a network.
- **Per-run PKI:** a Docker volume per run holds the CA, the stations' certificates and, with station key derivation, the vehicles' secrets. Each station mounts only its own directory and the public certificates. The volume is deleted with the run.

Trust boundaries

Party	Can see	Cannot see
Eavesdropper	frames on the message network	MQTT, control network, keys, ground truth
Station	its own keys and certificates, the public certificates	CA keys, other stations' files
Per-run PKI (serving)	CA keys, public material, batch state	vehicles' caterpillar and AT private keys (station key derivation)
Control broker clients	the topics their account allows	(one shared account per run; no TLS)
Service users	their own runs and runs shared with them	broker credentials, container settings outside the policy

Ground truth for scoring comes from the stations' own logs and the PKI's issue log, never from the eavesdropper's view.

Software

Part	Where	Role
Patch set	`patches/vanetza-nap/`	changes to Vanetza-NAP (patch documentation)
`vnapctl`	`sim/vnapctl`	command line; thin launcher for `vnapsim`
`vnapsim`	`sim/vnapsim/`	the simulation library used by `vnapctl` and the service
Helper images	`sim/images/`	PKI, mobility client, eavesdropper, control client, message checker
Service	`service/vnapapi/`, `service/ui/`	multi-user HTTP API and web UI

`vnapsim` modules:

Module	Contents
`scenario.py`	loading, `--set` overrides, validation, identity assignment, the user policy
`schema.py`	JSON Schema of the scenario format (`vnapctl schema`)
`lifecycle.py`	`up` / `down`, instance allocation, helper images, the run's PKI
`status.py`, `logs.py`	discovery, status, log parsing
`events.py`, `check.py`	event collection, metrics and expectations
`scoring.py`	eavesdropper links and tracks against ground truth
`cli.py`	the command line

The service imports `vnapsim` from `sim/` (`VNAP_SIM_DIR` overrides the location). It runs runs as jobs, gives each run a free instance, CPU and memory limits and its own broker credentials, and keeps results after a run stops.

Dependency on C-ITS-PKI

vnap-secure uses C-ITS-PKI in two ways:

Use	What	When C-ITS-PKI is needed
Per-run PKI (`[pki]` in a scenario)	The `vnap-pki` image is built from `sim/images/pki/` plus C-ITS-PKI's `src/` package. `pki_service.py` imports from it the CA hierarchy (`src.pki`), certificate issuing (`src.certificates`), butterfly key expansion and key handling (`src.crypto`), and the certificate types (`src.types`). C-ITS-PKI generates every certificate of such a run, including the butterfly ATs	when `vnap-pki` is built: on the first `vnapctl up` of a `[pki]` scenario, after any change to C-ITS-PKI's `src/`, or with `make pki-image`
Fixed certificate set (`certs/c-its-pki/`)	root, TLM, EA, AA, a regular AT and 24 butterfly ATs, made by C-ITS-PKI's `gen-vnap-certs.sh` and committed here	only to regenerate the set (certificates)

- **Submodule:** C-ITS-PKI is the git submodule `external/C-ITS-PKI` (URL `../C-ITS-PKI.git`, relative to this repository's), pinned to a tested commit.
- **Lookup order** in `vnapctl`, the service and `make pki-image`:
  1. `CITS_PKI_DIR`, e.g. a working copy you are changing;
  2. the submodule;
  3. a checkout next to vnap-secure (`../C-ITS-PKI`), for the transition, with a note on stderr.
- **Provenance:** the `vnap-pki` image, the run's PKI container and the service's `run.json` carry the C-ITS-PKI commit (label `vnap.cits_pki`, `-modified` when `src/` has uncommitted changes). `vnapctl status` shows it on the run PKI line.
- **Versions:** the `vnap-pki` image tag includes a digest of C-ITS-PKI's `src/`, so a change builds a new image on the next `[pki]` run, and two versions never overwrite each other.
- **Without C-ITS-PKI:** `vnapctl` uses the newest `vnap-pki` image already built. With no image either, `[pki]` scenarios (including the service's templates) fail validation with "C-ITS-PKI not found … run 'git submodule update --init'". Scenarios with fixed certificate pools need only `certs/`.
- **The contract:** C-ITS-PKI's `tests/test_vnap_secure_interface.py` checks the names and keyword arguments `pki_service.py` uses, and that butterfly private and public key expansion agree. Its Operations Guide §8 documents the same contract from its side.

Updating C-ITS-PKI:

```
cd ${VNAP_HOME}
git -C external/C-ITS-PKI fetch && git -C external/C-ITS-PKI checkout origin/main
cd ${VNAP_HOME}/sim && ./vnapctl up --instance auto refill && ./vnapctl --instance <n> check
./vnapctl --instance <n> down && cd ${VNAP_HOME}
```

```
git add external/C-ITS-PKI && git commit -m "Update C-ITS-PKI to $(git -C external/C-ITS-PKI rev-parse
    --short HEAD)"
```

Developing both at once: change C-ITS-PKI in its own checkout and point `CITS_PKI_DIR` at it while testing here. Push C-ITS-PKI first, then move the submodule pointer. Commits made inside `external/C-ITS-PKI` on a detached HEAD are easily lost: create a branch there first.

Dependency on vanetza-nap

The patch set applies to upstream vanetza-nap `release2-main` at the commit in `patches/vanetza-nap/BASE`. vanetza-nap is not a submodule:

- the patched commits are not published;
- its case-only file name differences rule out a checkout on a shared host folder;
- a recursive clone would carry about 140 MB of history that the service does not need.

`make image` fetches the base when needed (build).

Installation and build

Installation

docs/installation.md

How to prepare a development machine, from a fresh VM to the first build. The README has the short version.

Why a Linux VM

Vanetza-NAP has file names that differ only by case (e.g. `vanetza/asn1/its/r2/actionid.c` and `ActionID.c`). It cannot be checked out or built on a case-insensitive file system such as the default macOS one. Development therefore uses an Ubuntu 24.04 VM (VMware on a MacBook); every Docker image is built in the VM.

1. Install the prerequisite packages

On a fresh Ubuntu 24.04 VM, install `git`, `make`, `python3.12-venv` and Docker:

```
sudo apt update
sudo apt install -y git make python3.12-venv docker.io
sudo usermod -aG docker $USER       # then log out and in again, so docker runs without sudo
```

- `python3.12-venv` provides the `venv` module and pip for the virtual environment of step 5; Ubuntu's `python3` has neither.
- **Docker** from another source works too: Docker's own apt repository (`docker-ce`), or snap, with the limits in step 4.
- `jq` is optional: the monitoring and API examples in this documentation use it.

2. Prepare the VM

A full station image build needs about 10 GB of Docker build cache. Grow the VM's disk in VMware, then extend the root volume:

```
${VNAP_HOME}/scripts/vm/extend-docker-fs.sh      # lvextend -l +100%FREE -r on the root logical volume
```

The script is in the repository, so run it after step 3.

Without LVM, use `growpart`, `pvresize` and `lvextend -r` by hand.

3. Get the sources

```
INSTALL_DIR=<desired installation directory>
git clone --recurse-submodules git@github.com:jodyhuntatx/vnap-secure.git ${INSTALL_DIR}/vnap-secure
export VNAP_HOME=${INSTALL_DIR}/vnap-secure
```

- `INSTALL_DIR` **and** `VNAP_HOME` are used throughout this documentation: `INSTALL_DIR` is the directory the repository is cloned into, `VNAP_HOME` the repository itself. The scripts do not read them (each finds the repository from its own location). `export` lasts for the shell session: add the `export VNAP_HOME=…` line to `~/.bashrc` to keep it.
- **C-ITS-PKI** comes with it, as the submodule `external/C-ITS-PKI` at the tested commit. After a `git pull` that moved the submodule, or in a clone made without `--recurse-submodules`, run `make submodule` (`git submodule update --init`). See architecture.
- **vanetza-nap** is not cloned by hand: `make image` clones the upstream base into `~/vanetza-nap` (or `VANETZA_NAP_DIR`) the first time. See build.

4. Sharing the repository with the host (optional)

The repository can live on the host and be shared into the VM. vanetza-nap cannot: it must stay on the VM's own file system because of its case-only file name differences.

```
${VNAP_HOME}/scripts/vm/mount-shared-folder.sh     # in the VM: mount the VMware shared folders at
    /mnt/hgfs
ln -s /mnt/hgfs/<host-directory> ${INSTALL_DIR} # e.g., so the repository is at ${INSTALL_DIR}/vnap-secure
```

- **Git on** `/mnt/hgfs` reports "dubious ownership", because the mount shows the files as owned by another user. Use `git -c safe.directory='*' …`, or run `git config --global --add safe.directory <path>` once.
- **Snap-confined Docker** reads only files under the real `$HOME`, and not in dot-directories.
  - It cannot read build contexts on `/mnt/hgfs`. The scripts therefore send build contexts on stdin (`tar … | docker build -`), and `sim/images/msgcheck/build-msgcheck.sh` stages its files in `$HOME`.
  - Bind mounts from `/mnt/hgfs` do work: `vnapctl` mounts `certs/` from there.
  - vanetza-nap's build context (`~/vanetza-nap`) is in `$HOME`, as required.

5. Set up the Python virtual environment

The API service's tests need the packages of `service/requirements.txt`. They go into the virtual environment `service/.venv`, not into the system Python:

```
cd ${VNAP_HOME}
make service-deps                          # creates service/.venv (with pip) and installs
    requirements.txt
```

By hand, the same is:

```
cd ${VNAP_HOME}/service
python3 -m venv .venv                      # needs python3.12-venv (step 1); puts pip into .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

- `make test` **runs** `make service-deps` **first**, so the packages are in place before the service tests start. It installs again whenever `requirements.txt` changes.
- **A** `.venv` **without pip** ("No module named pip") was created while `python3.12-venv` was missing. Install the package (step 1) and run `make service-deps`: it creates the environment again.
- **Check:** `${VNAP_HOME}/service/.venv/bin/python -m pip list` shows `fastapi`, `uvicorn` and the others.

6. First build and smoke test

```
cd ${VNAP_HOME}
make image                                 # about 20 minutes the first time
make test                                  # unit tests, UI syntax, patch diffs, documentation links
cd ${VNAP_HOME}/sim && ./vnapctl up trust && ./vnapctl check && ./vnapctl down
```

`./vnapctl check` should end with `PASS`.

Build

docs/build.md

Station image

```
cd ${VNAP_HOME}
make image                                    # = scripts/build/docker-build.sh
IMAGE=vnap:r2-p20 make image                  # another tag
VANETZA_NAP_DIR=~/vanetza-test make image     # another vanetza-nap tree
make origs                                    # the unpatched upstream sources, for comparison
```

`scripts/build/docker-build.sh`:

1. **Base.** `patches/vanetza-nap/BASE` names the upstream repository, branch and commit (`release2-main` @ `241438fd`). If `VANETZA_NAP_DIR` (default `~/vanetza-nap`) does not exist, the script clones that commit there.
   - An existing tree must contain the base commit, for example upstream at `BASE` or a branch built on it. Anything else is refused, not reset.
2. **Patch set.** It copies `patches/vanetza-nap/modified/` over the tree: each file is named after its path with `/` written as `__`, e.g. `tools__socktap__time_trigger.cpp`. `make origs` copies `upstream/` instead, restoring the original files. Files the patch set adds stay.
3. **Build.** `docker build --network=host -t $IMAGE .` in the tree. The first build takes about 20 minutes and needs about 10 GB of build cache; later builds reuse the cached dependency layers. `NO_BUILD=1` stops after preparing the sources.

The vanetza-nap tree must be on the VM's own file system (see installation). A fresh clone with the patch set is byte-identical to the patched branch the current image was built from (checked during the reorganization; see CHANGELOG).

Check the result:

```
docker run --rm --entrypoint /usr/local/bin/socktap vnap:latest --help | grep -E
    'pseudonym-control|pki-refill'
```

Image tags

- **Tags:** `vnap:latest` is the newest build. Measurements use a fixed tag (`vnap:r2-pNN`), so results can name their image. Tag a build with `docker tag vnap:latest vnap:r2-pNN`, and add the tag to CHANGELOG.md.
- **Users of the service** may choose only the images in `sim/policy.toml` (`approved_images`). Add a new tag there when users should be able to select it.

Helper images

`vnapctl` builds these from `sim/images/` when a scenario needs them. Each is tagged with the digest of its sources (`<image>:<16 hex digits>`, plus `:latest`), so different checkouts never overwrite each other's images and a source change rebuilds on the next run.

Image	Sources	Used for
`vnap-pseudo-ctl`	`sim/images/pseudo-ctl/`	control broker and pseudonym event client
`vnap-mobility`	`sim/images/mobility/`	mobility client (routes, crossings, mix zones)
`vnap-eavesdropper`	`sim/images/eavesdropper/`	passive tracking attacker
`vnap-pki`	`sim/images/pki/` + C-ITS-PKI `src/`	the run's own PKI; the image label `vnap.cits_pki` records the C-ITS-PKI commit

- `make pki-image` builds `vnap-pki:latest` by hand (`sim/images/pki/build.sh`), with C-ITS-PKI found as `vnapctl` finds it: `CITS_PKI_DIR`, then the submodule.
- `make msgcheck` builds `vnap:msgcheck`, the offline message checker. It compiles `sim/images/msgcheck/vnap-msgcheck.cpp` against the same vanetza-nap tree, so build the station image first. See troubleshooting.

Patch review and diffs

```
make diffs                    # regenerate patches/vanetza-nap/diffs/ (one unified diff per file)
make diffs-check              # part of make test: diffs/ matches modified/ and upstream/
scripts/build/src-diffs.sh    # side-by-side view in a terminal
```

`diffs/` is generated: edit `modified/`, then run `make diffs`. What each patch does and why is in the patch documentation.

The same patches are committed as a series on the local vanetza-nap branch `jodyhuntatx`, which is not published. A tree on that branch already contains them, so `make image` on it changes nothing.

Tests

```
make test           # all of the following
make test-sim       # vnapsim unit tests (no docker)
make test-service   # service unit tests (no docker; installs service/requirements.txt first)
make test-ui        # JavaScript syntax of the web UI (node in a container)
make openapi-check  # docs/reference/openapi.json matches the API (regenerate with make openapi)
make diffs-check
make docs-check     # relative links and anchors in all Markdown files
make docs-pdf-check # docs/vnap-secure-documentation.pdf is current (rebuild with make docs-pdf)
```

Station behaviour is tested with live runs: a scenario, then `vnapctl check` with the scenario's expectations. C-ITS-PKI changes are tested from its side by `tests/test_vnap_secure_interface.py`, and here by a `[pki]` scenario (see architecture).

Documentation PDF

`docs/vnap-secure-documentation.pdf` holds all Markdown documentation in one file, with an introduction on the design intention of VNAP-Secure. After changing any Markdown file:

```
make docs-pdf       # rebuild the PDF (installs scripts/docs/requirements.txt into scripts/docs/.venv)
git add docs/vnap-secure-documentation.pdf
```

- **Kept current by `make test`:** the PDF carries a digest of its sources (every Markdown file, the introduction, `docs/images/`, the builder). `make docs-pdf-check` fails when they no longer match. The check needs only the standard library.
- **Reproducible:** the same sources give the same bytes, so rebuilding without a change leaves git clean.
- **Introduction:** `scripts/docs/pdf-intro.md`.
- **A new Markdown file** must be added to a part in `PARTS` in `scripts/docs/build-pdf.py`; the build and the check fail until it is. Links between documents become links within the PDF; links to other repository files point to GitHub.
- **Fonts:** DejaVu (`fonts-dejavu-core`, standard on Ubuntu). Italics use `fonts-dejavu-extra` when installed, otherwise reportlab's Vera.

Certificates

docs/certificates.md

Scenarios either bring their own certificate files (fixed sets in `certs/`) or get a fresh PKI per run (certificate refill). The fixed sets are **test material**: their private keys are committed and must never be used outside the simulation. `certs/README.md` lists the files and their validity.

Fixed sets

The repository's `certs/` is mounted read-only into every station at `/vnap-certs`. A scenario's `certs_dir` mounts another directory instead (see scenario format).

`certs/c-its-pki/` (v3, `certs-v3`)

Made with C-ITS-PKI:

- the authorities: `root_ca`, `aa`, `ea` and `tlm`;
- a regular AT (`at.cert` / `at.der`), used by RSUs;
- 24 butterfly ATs (`bke_at_<j>.cert` / `bke_at_<j>_sign.der`), used by fixed pseudonym pools.

The CA private keys are not kept: the set only has to verify. C-ITS-PKI's `docs/compliance.md` documents the certificates' conformance.

`certs/certify/` (v2, `certs-v2`)

Made with Vanetza's `certify` tool (TS 103 097 V1.2.1 format): a root and AA (`root_ca_vnap`, `aa_vnap`, certificate and key) and one AT per station (`ticket_vnap_rsu`, `ticket_vnap_obu`). Used by the `trust:v2` variant.

Validity and regeneration

Certificates expire, and a station drops messages signed under an expired AT without logging anything: the scenario then fails its check with "no CAMs received". Check the sets and make new ones with `vnapctl certs`, or in the web UI under *Administration* → *Test certificates* (administrators; web UI).

```
cd ${VNAP_HOME}/sim
./vnapctl certs                                              # every certificate with its validity; exit
    1 if one expired
./vnapctl certs regenerate v3 --lifetime at_hours=8760       # new v3 set, ATs valid for a year
./vnapctl certs regenerate v2 --lifetime at_days=90
```

Lifetimes. No ETSI document gives defaults: TS 103 097 fixes no certificate lifetime, and TS 102 941 leaves it to deployment policy (C-ITS-PKI's review of the standards, `external/C-ITS-PKI/docs/privacy-and-deployment-notes.md` section 2). The defaults are those of the tool that makes each set, and each can be changed:

Set	Lifetime key	Certificate	Default	Basis
v3	`root_years`	root CA	10 years	C-ITS-PKI default
v3	`tlm_years`	trust list manager	10 years	C-ITS-PKI default
v3	`ea_years`	enrolment authority	5 years	C-ITS-PKI default
v3	`aa_years`	authorization authority	5 years	C-ITS-PKI default
v3	`at_hours`	`at.cert` and the butterfly ATs	168 hours	C-ITS-PKI default
v2	`root_days`	root CA	365 days	`certify` default
v2	`aa_days`	authorization authority	180 days	`certify` default
v2	`at_days`	the station ATs	7 days	`certify` default

- **Short ATs expire quickly.** One week (the AT default of both tools) suits a deployment. For a committed test set it means regenerating every week, so pick a longer `at_hours` or `at_days` for test sets that should last.
- **Nesting:** a certificate may not outlive the one that signed it. An AA longer than the root CA, or ATs longer than the AA, are refused.
- **How it works:** the new set is built in a scratch directory under `certs/`. v3 is made by `sim/images/pki/test_certs.py` in the `vnap-pki` image, which `vnapctl` builds from the C-ITS-PKI submodule. v2 is made by the steps of `scripts/certs/gen-certify.sh` with `certify` in `vnap:latest`. The new set is read back and checked for missing or expired files. Only then are the set's `*.cert`, `*.der` and `*.key` files replaced. The new files keep the same names, so scenarios keep working.
- **Record:** `generated.json` in the set holds the time, who regenerated it, the lifetimes and, for v3, the C-ITS-PKI commit.
- **Afterwards:** the files are tracked in git, so commit them. Running simulations keep the certificates they loaded; new runs use the new ones.
- **Reading validity:** v3 dates come from C-ITS-PKI's decoder with Vanetza's unit lengths (a year is 31,556,952 s). v2 dates come from `certify show-certificate`, the parser the stations use.

C-ITS-PKI's `gen-vnap-certs.sh` and `scripts/certs/gen-certify.sh` still work by hand, but they take no lifetimes for the authorities and write no record.

Key formats

socktap loads private keys as PKCS#8 DER (v3 also accepts PEM). Keys from `certify` and C-ITS-PKI already have that format. To convert a PEM key:

```
scripts/certs/convert-sign-key.sh <key.pem>     # writes <key.pem>.der and prints it
```

Trust configuration

The container environment sets the certificates: `AT_CERT` / `AT_KEY` (or a pseudonym pool), `AA_CERT` and `ROOT_CERT`. In scenarios, the station and default keys `at_cert`, `at_key`, `aa_cert`, `root_cert` and `pseudonyms` set them. The negative controls point `root_cert` at the wrong certificate on purpose (`trust:badroot` trusts `tlm.cert`). How the stations check chains is described in security and chain verification.

Test certificates

certs/README.md

Test material only. The private keys in this directory are public: they are committed to git. Never use these certificates or keys outside the simulation.

`vnapctl` mounts this directory read-only into every station at `/vnap-certs`; scenario files refer to it as `/vnap-certs/...`. `vnapctl certs` shows every certificate's validity, and `vnapctl certs regenerate v2|v3` (or the web UI's administration page) makes a new set with chosen lifetimes: see docs/certificates.md. A regenerated set has a `generated.json` with the lifetimes used.

Directory	Format	Made with	Files
`c-its-pki/`	v3 (`certs-v3`), TS 103 097 V1.3.1	C-ITS-PKI	`root_ca`, `tlm`, `ea`, `aa` certificates; regular AT `at.cert` / `at.der`; butterfly ATs `bke_at_<0..23>.cert` / `bke_at_<j>_sign.der`
`certify/`	v2 (`certs-v2`), TS 103 097 V1.2.1	Vanetza `certify`	`root_ca_vnap`, `aa_vnap` (certificate and key); ATs `ticket_vnap_rsu`, `ticket_vnap_obu` (certificate and key)

Scenarios with their own PKI (`[pki]`) do not use these files: their certificates are created per run and deleted with it.

How it works

Security and chain verification

docs/functional/security-and-chain-verification.md

Stations sign every message with an authorization ticket (AT) and verify received messages, following ETSI TS 103 097: `certs-v2` for V1.2.1 certificates, `certs-v3` for V1.3.1 / IEEE 1609.2 certificates.

Trust chain

```
root CA ──signs──▶ AA (authorization authority) ──signs──▶ AT (pseudonym certificate) ──signs──▶ CAM
```

Each station is configured with:

- its own AT and key, or a pool of ATs;
- the AA certificate (the chain);
- the trusted root.

A received message is accepted only if its signer's AT chains, through an AA, up to the trusted root, and the signature verifies. Upstream v3 accepted any AT regardless of its issuer; the patch set adds the full check, AT → AA → trusted root, with the IEEE 1609.2 signing input. It also makes `--trusted-certificate` work for v3.

Certificate checks in the station log

At startup, each station checks its configured chain and its own AT(s), and logs the results:

```
[V3-CHAIN] chain certificate /vnap-certs/c-its-pki/aa.cert: OK
[V3-CHAIN] own authorization ticket /vnap-certs/c-its-pki/at.cert: OK
```

- **Refill:** with certificate refill, every AT that arrives during a run is checked the same way before it is used, and rejected if the chain or its key does not match:

  ```
  [V3-CHAIN] batch authorization ticket /tmp/vnap-pki/2-2/7.cert: OK
  ```

  Here obu1 (configured station ID 2) received ticket 7 of its 2nd batch request (`2-2`), and its signature chain to the root CA is valid.
- **Where the checks show up:** `vnapctl events --kind chain` lists these "certificate check" events, and `status` and the web UI report a failed one as a warning.
- **Expectations:** the default expectations require `<station>.chain.fail == 0`.

Verification results

- **Results:** each received message is published on the station's MQTT broker with `security_report` (`Success`, or the failure reason such as `Invalid_Certificate`).
- **Rates:** `check` computes per-station success rates from these, and expects a CAM success rate of 1 with security enabled (vnapctl).
- **Non-strict decapsulation:** socktap delivers messages even when they fail verification.
- **Negative controls:** `trust:badroot` (trusts the TLM certificate instead of the root) and `controls/naive-v3` (self-made certificates) must show failing reports; their expectations require exactly that.

Robustness

Verification runs on several reception threads while another thread signs. Several upstream data races there crashed socktap under load. They were found with ThreadSanitizer and fixed:

- the sign header policy;

- the certificate caches;
- the runtime clock and ASN.1 descriptors;
- the routers, PubSub/MQTT/DDS and the RSSI reader.

See thread safety. `controls/stress-naive-v3` (4 stations at 50 Hz) reproduces the load.

Related

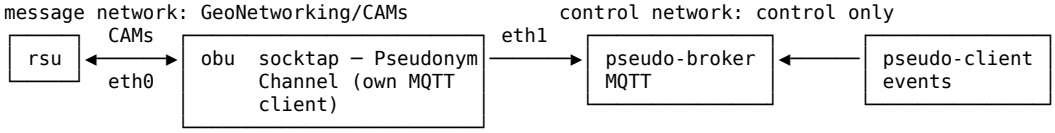
- **Certificates:** the fixed test sets and key formats (certificates).
- **Patches:** certificate verification.
- **Limitations:** the known upstream limitations are in troubleshooting.

Pseudonym change

docs/functional/pseudonym-change.md

A station with a pseudonym pool keeps its current pseudonym (authorization ticket and signing key) until it receives a change event. A change is a synchronized change of all of the station's identifiers, optionally followed by a silent period, so that an eavesdropper cannot simply follow the station across it.

Event-driven changes



- **Channel:** a dedicated MQTT connection from socktap (`tools/socktap/pseudonym_channel.cpp`) to the control broker on the control network. Events, actions and answers are specified in control channels.
- **Who sends events:**
  - the event client (periodically, at random intervals, once, or by hand);
  - the mobility client, when a vehicle enters a mix zone or enough vehicles are near each other (vehicle movement);
  - the service's control actions.
- **Thread safety:** the MQTT thread only queues the event. The change runs on socktap's `io_context` under the `TimeTrigger` mutex, which also guards CAM signing, so a change cannot fall between the certificate and private key lookups.
- **After a change:** the next signed message carries the full new certificate, so receivers learn it at once.
- **Pool:** a fixed pool (scenario `pseudonyms = { cert, key, count }`) wraps around after its last pseudonym. With certificate refill, the pool grows at runtime and no pseudonym is used twice.

ID change notification (ETSI TS 102 723-8 / -9)

With `id_change = "full"` (`PSEUDO_ID_CHANGE=full`, the default), a pseudonym change follows the two-phase commit of clause 6.3:

1. **Subscribers:** the network and transport layer (GeoNetworking routers) and the facilities layer (CAM application) subscribe to the security entity's ID change service (`vanetza/security/id_change_service`).
2. **PREPARE** goes to all subscribers, with the 8-octet id of the new authorization ticket (its HashedId8). The network layer locks every router, so nothing is sent with the old identifiers until the change completes.
3. **The security entity switches** to the new authorization ticket and key.
4. **COMMIT:**
   - every router gets a new GN address, whose MID is also the source MAC of its frames;
   - the CAM application uses a new `stationId`;
   - both are derived from the id with SHA-256 under separate labels, so they share no bytes with each other or with the certificate.

   If a subscriber refuses PREPARE, everyone prepared gets **ABORT** and nothing changes.

Service	Clause	Here
IDCHANGE-SUBSCRIBE / -UNSUBSCRIBE	5.2.5, 5.2.7	`IdChangeService::subscribe()`; socktap's subscriptions unsubscribe on destruction
IDCHANGE-EVENT (PREPARE, COMMIT, ABORT, DEREG)	5.2.6, 6.3.1	hook functions; DEREG when the security entity is destroyed
IDCHANGE-TRIGGER	5.2.8	`trigger()`: the security entity changes to the next pseudonym; control action `trigger`
ID-LOCK / ID-UNLOCK	5.2.9, 5.2.10	`lock(seconds 0..255)` / `unlock(handle)`; changes are refused while locked; control actions `lock` / `unlock`

- **Logging:** each change is logged as `[IDCHANGE] network: GN address MID / MAC <old> -> <new>` and `[IDCHANGE] facilities: stationId <old> -> <new>`, plus ID-LOCK and ID-UNLOCK lines. `vnapctl` uses these lines to keep naming a station whose IDs change: `status` shows its current identity, and `events --kind idchange` the changes.
- **Certificate-only changes:** `id_change = "certificate"` (`PSEUDO_ID_CHANGE=certificate`) changes only the certificate. This is useful as a comparison: the eavesdropper then links every change through the unchanged identifiers.
- **Not changed:** the configured station ID and MAC remain the station's identity on its local MQTT interface (e.g. `receiverID`) and its control topics. Messages injected on `vanetza/in/*` carry whatever `stationId` their payload has.

Silent period

With `pseudonyms.silent_min_ms` / `silent_max_ms` (`PSEUDO_SILENT_MIN_MS` / `PSEUDO_SILENT_MAX_MS`), the station sends nothing for a random time in that range after each full ID change. This is the silent period strategy of ETSI TR 103 415 clause 4.1.4; clause 4.2.1 reports the SAE J2735 values of 3 to 13 s.

- **How:** the network layer starts it on COMMIT, and `DccPassthrough` drops all outgoing frames (CAMs, beacons, injected messages) until it ends. Reception continues.
- **Logging:** `[IDCHANGE] silent period <n> ms` at the start, and `silent period over, <n> frame(s) suppressed` at the end.
- **Cost** (TR 103 415): while silent, the vehicle is missing from its neighbours' view, and it reappears suddenly afterwards.

CAM timing

The CAM timer restarts at a random phase on every full ID change. The first CAM after the change is skipped, and the next comes after a random delay of up to one interval (`[IDCHANGE] CAM timer: new phase, next CAM in <n> ms`).

Without this, each car's position within the 1 s CAM cycle carried over to its new identity and linked the two: a timing-only linker matched 16 of 16 changes in the random-turn mix zone, and 1 of 13 with the random phase (eavesdropper).

Security of the channel

Whoever can publish change events controls when a station changes pseudonym. See control channels.

Related

- **Scenario keys:** `pseudonyms` per station, `[control]`, `[control.client]` and `[[control.mix_zones]]` (scenario format).
- **Patches:** pseudonym and ID change, mobility and CAM.
- **Results:** the mix zone and convoy test summaries in results.

Certificate refill (per-run PKI)

docs/functional/certificate-refill.md

Instead of a fixed set of certificates that every run reuses (and that wraps around when used up), each run can have its own certificate authority. Stations start with a batch of butterfly authorization tickets (ATs) and request a new batch when only a few unused ones are left. By default the stations derive their ATs' private keys themselves (butterfly key expansion), so the PKI never holds them. The design is in the product requirements document (Specs/VNAP-Secure-Simulation-Service-PRD.pdf, sections 5.5 and 7).

```
cd ${VNAP_HOME}/sim
./vnapctl up refill                # run's CA, RSU AT, 8 butterfly ATs per OBU; refill at 2 unused
./vnapctl status                    # per OBU: unused ATs, batches, refresh time; the PKI's issue times
./vnapctl events --kind pki --since 2m
./vnapctl check --expect 'obu1.pki.refresh_ms_max<2000'
./vnapctl down                      # also deletes the run's PKI volume with all its keys
```

Scenario keys

- `[pki]`**:**
  - `initial` (8), `refill_at` (2), `batch` (8), `validity_hours` (24);
  - `etsi_version` (default: from the stations' security);
  - `ip` (default: host .5 of the control network);
  - `key_derivation`: `station`, the default, or `pki`, where the PKI makes and returns the private keys.

  It needs a `[control]` section.
- **Per station:** `pseudonyms = { initial, refill_at, batch, ... }` overrides the defaults. `refill_at = 0` gives a fixed pool from the run's PKI that wraps around.
- **Regular ATs:** stations with certificates but no `pseudonyms` get one.
- **No certificate paths:** `at_cert`, `aa_cert`, `root_cert` and `pseudonyms.cert` are not allowed with `[pki]`. The fixed-file pools remain available in scenarios without `[pki]`.

What up does

1. Creates a Docker volume for the run and provisions it.
2. Starts the PKI container (`pki`, role `pki`) on the control network.
3. Mounts into each station only `public/` and its own `stations/<name>/`, read-only. A station sees neither the CA keys nor the other stations' files.

A failed start removes the volume again; `down` removes it with the run.

With `key_derivation = "station"`, provisioning is a separate one-shot container. The serving PKI mounts only `private/` and `public/`, so it cannot read the vehicles' caterpillar keys or AT keys either. With `pki`, one container provisions and serves.

PKI service

`sim/images/pki/`, image `vnap-pki`, built from that directory and C-ITS-PKI's `src/` (architecture):

- `provision`**:** creates the run's root CA, TLM, EA and AA, and a regular AT for road-side units. For pseudonym stations it also does the enrolment (caterpillar keys) and issues an initial batch of butterfly ATs.
- `serve`**:** answers batch requests on the control broker; each batch uses the station's next i-period. It logs issuance and queueing time per request.
- **Volume layout:**

- `public/`: the root, AA and other certificates.
- `stations/<name>/`: the station's own files. With station key derivation this includes the vehicle's caterpillar private key, expansion key and enrolment key.
- `private/`: never mounted into stations. It holds:
  - the CA keys;
  - per station, the caterpillar public key (or, with `key_derivation = "pki"`, the caterpillar keys);
  - the expansion key, the enrolment certificate and the batch state.
- `private/issued.jsonl`: every issued AT with its HashedId8, the ground truth for scoring. The service keeps it (digests only) with a run's results.

Key derivation on the station

Butterfly key expansion (IEEE 1609.2.1) splits the work so that the PKI never sees the ATs' private keys:

- **The PKI** uses only the vehicle's caterpillar public key A and expansion key k. For each AT (i-period i, index j) it computes the cocoon key A + f_k(i, j)·G, certifies cocoon key + r·G, and returns the certificates with the offsets r and indices j, without keys.
- **The station** computes each private key as a + f_k(i, j) + r mod n, with f_k the IEEE 1609.2.1 expansion function as implemented in C-ITS-PKI (`tools/socktap/bke.{hpp,cpp}`). It checks every key against its certificate and rejects ATs whose key does not match.
- **The initial batch** is derived once at provisioning, playing the vehicle; those keys are written only to the vehicle's directory.
- **Options:** `--pki-caterpillar-key`, `--pki-expansion-key` (`PKI_CATERPILLAR_KEY`, `PKI_EXPANSION_KEY`).

Stations

Options `--pki-refill-at`, `--pki-batch-size`, `--pki-topic`, `--pki-retry` (`PKI_REFILL_AT`, `PKI_BATCH_SIZE`, `PKI_TOPIC`):

- **Requests:** when a change leaves `PKI_REFILL_AT` or fewer unused ATs, the station requests a batch on the control broker and installs the answer while running (control channels). Each AT is chain-checked first (`[V3-CHAIN] batch authorization ticket ...`).
- **Retries:** at most one request is outstanding; unanswered requests are repeated after 10, 20, 40 … seconds (up to 120).
- **No reuse:** with refill on, no AT is used twice. With none left, changes are refused (`[PSEUDONYM] change rejected: no unused pseudonym left`) until a batch arrives.
- **Logs:**
  - `[PKI] batch requested: request <id>, <u> unused, <n> wanted`;
  - `[PKI] batch installed: request <id>, <n> certificate(s), refresh <ms> ms (PKI issue <ms> ms, key derivation <ms> ms), <u> unused`.

  Refresh time runs from the first request for a batch (retries included) to installation.

Status, events and checks

- `status`:
  - per station, a refill line: unused ATs, batches, refresh mean and max, retries, starved changes, pending request;
  - a line for the run's PKI: provisioning time, batches, issue and queue times, and the C-ITS-PKI commit.

- `events --kind pki`**:** requests, installed batches and retries from the stations, and issued batches from the PKI.
- `check` **metrics:**
  - per station: `<station>.pki.batches`, `.added`, `.unused`, `.retries`, `.refused`, `.starved`, `.pending`, `.refresh_ms_mean`, `.refresh_ms_max`, `.derivation_ms_mean`;
  - for the PKI: `pki.running`, `pki.batches`, `pki.refused`, `pki.issue_ms_max`, `pki.queue_ms_max`.

  The default expectations add `<station>.pki.starved==0` and `pki.running==1`.

Sizing

Measured (`results/pki-refill-test-2026-10-07.docx`):

- The PKI issues about 50 ms per ticket.
- The refresh time is roughly the issue time × the batch size × the number of stations asking at once.
- A station starves when the refresh time exceeds (`refill_at` + 1) × the interval between its pseudonym changes. Synchronized changes, as in a mix zone, make stations ask at the same time.

Without vnapctl

The service can be provisioned and attached by hand, e.g. for another harness:

```
# provision into a host directory and use it as the scenario's certs_dir
docker run --rm -v ~/vnap-pki-runs:/runs vnap-pki provision --dir /runs/run1 --config '{"etsi_version":
    "v3",
  "stations": [{"name": "rsu", "station_id": 1, "certificates": "regular"},
               {"name": "obu", "station_id": 2, "certificates": "bke", "initial": 8, "batch": 8}]}'
# scenario: certs_dir = "~/vnap-pki-runs/run1" (absolute), aa_cert /vnap-certs/public/aa.cert,
#   root_cert /vnap-certs/public/root_ca.cert, OBU pool /vnap-certs/stations/obu/bke_at_{i}.cert
#   with key bke_at_{i}_sign.der and count 8, env PKI_REFILL_AT = "2"
docker run -d --name pki --network vnapctl0 -v ~/vnap-pki-runs:/runs -e CONTROL_BROKER=pseudo-broker \
  vnap-pki serve --dir /runs/run1
```

Related

- **Patches:** certificate refill.
- **Results:** `results/pki-refill-test-2026-10-07.docx`.

Vehicle movement

docs/functional/vehicle-movement.md

Upstream socktap has a fixed position (`config.ini`, or `VANETZA_LATITUDE` / `VANETZA_LONGITUDE`) or reads one from gpsd. With `--position-control-broker` (`POSITION_CONTROL_BROKER`), the position becomes controllable at runtime, and a mobility client drives the vehicles.

Position control channel

- **Station side:** socktap subscribes to `vnap/position/<station id>` on the control broker, on its own MQTT connection. Each update replaces the position fix that feeds:
  - the CAM: reference position, heading, speed, acceleration, yaw rate;
  - the GeoNetworking position vector of every packet.

  Updates go to the configured station ID, so a station keeps its topic across ID changes.
- **Payload and access:** see control channels. The static position provider is replaced by a thread-safe controllable one.

Mobility client

`sim/images/mobility/`, image `vnap-mobility`, container `mobility`:

- **Routes:** each vehicle waits at its first waypoint until `start_s`, then drives the polyline at constant speed. It publishes at `rate_hz` (default 5) with the retain flag. With `loop` it drives back to the first waypoint and starts over; without, it stops at the last one.
- **Random turns:** instead of a `route`, `mobility = { crossing = [lat, lon], arm_m = 150, start_arm = "east", speed_kmh = 36 }` drives laps through a four-way crossing whose arms end on a square ring road. Each lap is a random turn at the crossing (no U-turns), a random way along the ring, and back in on the next arm.
  - Every lap is 4 × `arm_m`, so vehicles keep their relative timing.
  - The choices come from `control.mobility_seed`. `vnapctl` picks and records one if unset; `status` shows it, so a run can be repeated.
- **Logging:** every vehicle's position every 10 s, and each lap's choices (`docker logs mobility`).
- **Scenarios:** a station's `mobility` key gives its route or crossing. `vnapctl` then:
  - connects the station to the control network;
  - starts it at the first waypoint;
  - starts the mobility client.

  The scenario needs a `[control]` section (scenario format; examples: `linkability`, `convoy`, `crossing`).

Mix zones

A mix zone is a place where several vehicles change pseudonym at about the same time and place, so that an observer cannot tell which new identity belongs to which old one.

The problem. A vehicle broadcasts CAMs several times a second with its position, speed and heading. Changing the pseudonym certificate alone hides little: when one car changes on an empty road, an eavesdropper sees one identity stop and another appear a moment later on the same trajectory, and links them. A change protects privacy only when there are other candidates for the new identity.

How a mix zone creates that ambiguity:

- **Many vehicles together:** every vehicle entering the zone changes there, so several old identities disappear and several new ones appear in the same small area.
- **Unpredictable movement:** the zone is where trajectories diverge, typically an intersection, so extrapolating a vehicle's heading does not single out one new identity.
- **Silence:** vehicles stop transmitting for a random time after the change, so the observer has a gap in position and in message timing instead of a continuous track.

The privacy gained is roughly the number of vehicles that could plausibly be the same one (the anonymity set). A vehicle alone in a zone gains nothing.

In the simulation:

Element	Role
`[[control.mix_zones]]`	the zone: the mobility client sends a change event to each vehicle entering it, which stands in for a vehicle knowing it is in a designated zone
`silent_min_ms`, `silent_max_ms` (`pseudonyms`)	the random radio silence after a full ID change (pseudonym change)
crossing mobility with random turns	unpredictable exits (`crossing`). With fixed turns (`crossing:blocks`) the eavesdropper's mistakes form a pattern an attacker could undo
`min_vehicles`	the "many vehicles together" condition: a vehicle does not spend a pseudonym on a change that would be linked anyway
the eavesdropper and its score	the test of the zone: it links old and new identities by position and CAM timing, and the score against ground truth says how well the zone worked (eavesdropper and scoring)

The mix zone is the countermeasure, the eavesdropper measures it, and the parameters above are what to vary to see when linking fails.

Configuration. `[[control.mix_zones]]` (name, center, radius, stations) makes the mobility client send a pseudonym change event to a vehicle each time it enters the zone, instead of on a clock. Set `control.client.mode = "manual"` for that. The events carry the reason `mix-zone <name>` and show up in `vnapctl events`.

Threshold. `min_vehicles = n` makes the change conditional on company in the zone (examples: `crossing:min3`, `crossing:blocks-min3`):

- A vehicle that enters waits while fewer than `n` vehicles are inside.
- When the `n`-th vehicle is inside, every waiting vehicle gets its change event in the same position update, so they change together.
- While `n` or more stay inside, a later arrival changes on entry.
- A vehicle that leaves before the threshold is reached keeps its pseudonym; the mobility client logs `left mix zone <name> without a change`.

Only the stations the zone watches count (`stations`, by default every station with mobility and pseudonyms), so an RSU at the crossing does not. A vehicle changes at most once per visit, and one that starts inside the zone counts but does not change. The default `min_vehicles = 1` changes every vehicle on entry.

Dynamic mix zones

A dynamic mix zone is a mix zone that travels with the vehicles: instead of a fixed circle on the map, the condition is that enough vehicles are near each other, wherever they are.

- **Rule:** a participating vehicle gets a pseudonym change event when at least `min_vehicles` participating vehicles, itself included, are within `radius_m` of it.
- **Cooldown:** at most one such change per vehicle every `cooldown_s`. The first one comes `cooldown_s` after the start at the earliest. Vehicles that stay together (a convoy) change again each time the cooldown ends.

- **Together:** vehicles that meet with the same threshold change in the same position update. With different thresholds, the one with the lower threshold may change alone.
- **Events** carry the reason `dynamic-mix-zone`. Set `control.client.mode = "manual"` so that no clock-driven changes mix in.

Two scopes select who takes part and with which threshold:

Scope	Where	Effect
Run	`[control.dynamic_mix_zone]` with `enabled`, `min_vehicles`, `radius_m`, `cooldown_s`	every station with mobility and pseudonyms takes part (`enabled = false`: only the stations that say so)
Station	`dynamic_mix_zone = { enabled = …, min_vehicles = … }` in a station	this station takes part or not, with its own threshold; replaces the run scope for it

Example: `convoy` (three cars, all three within 80 m, every 20 s). Fixed mix zones and a dynamic one can be used together.

It is an oracle. The mobility client knows every vehicle's true position, so vehicles react to ground truth, not to what they heard on the radio. It shows whether proximity-based changes help at all; a vehicle deciding from the CAMs it receives would be a change to the station.

Origin

Stations without a position, and the example layouts, are placed around the crossing of Rua Alexandre Herculano and Rua Venâncio Rodrigues in Coimbra (40.208106, −8.4197756; `ORIGIN` in `sim/vnapsim/common.py`). The crossings, arms and ring roads are synthetic geometry in metres around that point. They do not follow the real streets; for realistic paths, draw routes along streets in the web UI's builder.

Watching movement

```
# positions in the CAMs the RSU receives
docker run --rm --network vanetzalan0 eclipse-mosquitto:2 mosquitto_sub -h 192.168.98.10 -t
    vanetza/out/cam \
  | jq -c '.fields.cam.camParameters | [.basicContainer.referencePosition.latitude,
           .basicContainer.referencePosition.longitude,
           .highFrequencyContainer.basicVehicleContainerHighFrequency.heading.headingValue,
           .highFrequencyContainer.basicVehicleContainerHighFrequency.speed.speedValue]'
```

The web UI's *Map* tab shows the layout and live positions, and moves a station on click.

CAM kinematics

Upstream filled these fields from a static position only, and several were wrong in units once the vehicle moves. The patch set fixes them:

- **Heading:** was degrees copied into a field in 0.1°.
- **Speed and heading confidence:** were m/s and degrees copied into fields in 0.01 m/s and 0.1°. A confidence of 0.1 m/s became the invalid value 0, and the OBU could not encode its CAMs (`Can't determine size for unaligned PER encoding of type CAM because of SpeedConfidence`).
- **Longitudinal acceleration and yaw rate:** had the wrong scale. The yaw rate also had the wrong sign: ETSI counts it positive to the left, while the heading grows clockwise.

The JSON on `vanetza/out/cam` shows the decoded physical values (m/s, degrees).

Related

- **Patches:** mobility and CAM.
- **Results:** the convoy and mix zone results are in results.

Eavesdropper and scoring

docs/functional/eavesdropper-and-scoring.md

`sim/images/eavesdropper/` is a passive listener playing an untrusted party that wants to track vehicles. It joins only the message network with a raw socket (`NET_RAW`) and sees nothing but the frames on the wire: no keys or trust store, no MQTT, no control channel. It knows the public ETSI/IEEE formats.

What it decodes

Layer	Standard	Extracted
Ethernet		source MAC
GeoNetworking	EN 302 636-4-1	GN address (station type, MAC-derived ID), source position, speed, heading
Security header v2	ETSI TS 103 097 V1.2.1 (own parser)	signer, generation time, ITS-AID
Security header v3	IEEE 1609.2 / TS 103 097 V1.3.1 (OER, `asn1tools`)	signer, generation time, PSID
Signer certificate	same	HashedId8 digest, computed exactly as Vanetza does (SHA-256 over the canonical encoding); also issuer (AA), validity, permissions. Full certificates are sent about once a second
BTP	EN 302 636-5-1	port, message type
Facilities	TS 103 900 (CAM R2)	`stationId` of any message; full CAM: `referencePosition`, speed, heading, vehicle size, station type

The ASN.1 modules come from Vanetza-NAP (`sim/images/eavesdropper/asn1/`, with their source note).

Linking techniques

The eavesdropper groups messages into tracks (one per vehicle, as far as it can tell) and tries to carry a track across each pseudonym change. A new certificate digest is a new pseudonym; the first technique below that applies links it to an existing track.

Technique	Evidence used	Option (scenario key)	Defeated by
Shared identifiers	the new pseudonym's frame carries a MAC, GN address or `stationId` already seen on the track	always on	full ID change: MAC, GN address and `stationId` change with the certificate (`id_change = "full"`, the default)
Timing phase	CAMs come at a regular interval. The track's last CAM generation times (security header, 1 ms resolution) give its interval (least-squares fit) and phase; extended over the gap, they predict when its next CAM would come. One silent track must predict the new pseudonym's first CAM within the tolerance, and no other within twice the tolerance	`--link-by timing`, `--timing-window` 20 s, `--timing-tolerance` 25 ms (`link_by`, `timing_window`, `timing_tolerance_ms`)	restarting the CAM timer at a random phase on every ID change (pseudonym change)
Position continuity	the new pseudonym appears where a track that went silent could have driven in the gap: within `--link-distance` plus last speed × gap; the nearest one wins	`--link-by position`, `--link-window` 3 s, `--link-distance` 50 m (`link_by`, `link_window`, `link_distance`)	a silent period longer than the attacker waits, with other vehicles nearby that could be the one that reappears (convoys, mix zones with random turns)

- **Precedence:** shared identifiers link immediately. Timing beats position when it is unambiguous; its log entry then also says whether position agreed (`position pointed to another track` otherwise).
- **Confirmation:** timing and position links are confirmed only if the old identity stays silent for 1.5 message intervals, so two vehicles side by side are not merged.
- **Greedy:** each new pseudonym is decided on its own, the moment it appears. A wrong link uses up the old track, so the car that really owned it can no longer be linked.
- **Chance timing matches:** with random phases, a new pseudonym's first CAM sometimes lands within the tolerance of another car's prediction. In a 3-minute run with 4 cars that gave 1–2 timing links, about half of them wrong. An attacker sees the same coincidences.
- **Logging:** every linked change is logged with its evidence:
  - `same mac`, `same gn`, `same station`;
  - `timing phase (<n> ms off after <k> CAM interval(s), next <m> ms)`;
  - `position continuity (<d> m, <s> s gap; ...)`.

  The summary line, `tracks.json` and `vnapctl status` count the links per technique.
- **Not used (yet):**
  - heading and turn statistics;
  - multiple hypotheses, or a joint assignment of all changes at a crossing;
  - vehicle attributes in CAMs (length, width, station type);
  - the timing of GeoNetworking beacons and other periodic messages;
  - RSSI or several receivers.

Running it

```
cd ${VNAP_HOME}/sim
./vnapctl up crossing                  # four cars, random turns, mix zone, eavesdropper
./vnapctl status                        # includes the eavesdropper's tracks and linked changes
./vnapctl check --expect 'eavesdropper.linked_changes==0'   # a privacy goal
docker logs -f eavesdropper
images/eavesdropper/run.sh [network] [name]                 # attach to any running simulation
python3 images/eavesdropper/eavesdropper.py --pcap capture.pcap --log-dir out/   # offline (needs
    asn1tools)
```

- **In scenarios:** an `[eavesdropper]` section adds it (scenario format).
- **Metrics:** `vnapctl check` exposes:
  - `eavesdropper.frames`, `.decode_errors`, `.tracks`, `.pseudonyms`, `.linked_changes`;
  - per technique, `.linked_by_identifier`, `.linked_by_timing` and `.linked_by_position`.
- **Re-analysis:** `eavesdropper.py --replay messages.jsonl --log-dir out/ --link-by position` reruns the linking on a saved run's frame log with other options (no `asn1tools` needed). Get the log with `docker cp eavesdropper:/logs - | tar -x`.

Output	Content
`docker logs -f eavesdropper`	new tracks, linked pseudonym changes, periodic summary
`/logs/messages.jsonl`	every decoded frame
`/logs/events.jsonl`	track and linkage events
`/logs/tracks.json`	per track: identifiers, pseudonyms (first/last seen, certificate fields), last position, recent trail; rewritten every 5 s

Scoring against ground truth

`sim/vnapsim/scoring.py` compares the eavesdropper's conclusions with what really happened:

- **Ground truth:** which certificate each station actually used, from the stations' own logs (and, with a per-run PKI, its issue log).

- **When:** the service scores every run at stop (`score.json`) and shows the result live and on the *Results* tab. `GET /api/runs/<id>/eavesdropper` returns it.

Measure	Meaning	How to read it
Links (correct / wrong, per technique)	each pseudonym change the eavesdropper linked; correct when both pseudonyms belong to the same vehicle	wrong links are the attacker's mistakes; correct ones are privacy failures
Longest chain followed	the most pseudonym changes through which one track followed one vehicle without a mistake (consecutive identities of the same vehicle)	the privacy result in one number: 0 means no vehicle was followed through any change
Identities (per track)	pseudonyms the eavesdropper put into the track	1 means nothing was linked to it
Most Frequent (per track)	share of the track's pseudonyms that belong to its most frequent vehicle, and that vehicle: `75 % (obu1)`. In `score.json`: `purity` and `main_station`	read with Identities: many identities at a high share means that vehicle was tracked; a low share means vehicles were mixed up (the mix zone worked); 1 identity is always 100 %
Station predictions in actual order across identity changes (per track)	the real vehicle behind each pseudonym of the track, in the order the eavesdropper added them: `obu1 → obu1 → obu2`. In `score.json`: `stations`	one name throughout means that vehicle was followed; each change of name is a wrong link
Followed (per track)	pseudonym changes through which this track stayed on one vehicle	the share can overstate tracking: a track alternating between two cars is 50 % one car but follows neither

Related

- **Results:** what the eavesdropper achieved against each scenario, with recommendations for further scenarios, is in results.
- **Countermeasures:** the countermeasures it is measured against are described in pseudonym change.

Summary report

When a run stops, the service writes `report.docx` into the run's results, in the form of the test summaries in results. Download it on the *Results* tab or with `GET /api/runs/<id>/results/report.docx`.

Section	Content
Header table	date and duration, run, mobility seed, station image, security and PKI, eavesdropper, health
1. Purpose	the run's description
2. Setup	every station with its role, movement and pseudonym pool; what triggered ID changes; the silent period, configured and measured; the eavesdropper's settings
3. Results	pseudonym changes, mix zone entries, time silent, certificate refill, links (correct and wrong, per technique), tracks; then per vehicle, every link scored against ground truth, wrong links by vehicle pair, what each track contains (with the same *Most Frequent* and *Station predictions in actual order across identity changes* columns as the *Results* tab), observations
4. Conclusion	the result in a few sentences, with the chance level for comparison
5. Limitations of this run	those that follow from the run's setup
6. Suggested next runs	suggestions chosen by rule, see below

- **Facts, and rules:** sections 1 to 5 state what was measured. Section 6 is chosen by fixed rules from those measurements, not by judgement; an insight that nobody has written down as a rule stays with the reader.
- **Without an eavesdropper** the report has the setup and the counts, and no linkage.
- **Pattern of the mistakes:** from 8 wrong links on, the report says how many of them one fixed rule ("old vehicle X is really new vehicle Y") explains. At 80 % or more an attacker who knows the traffic pattern could undo most of them.
- **Source:** only the files in the run's results (`run.json`, `status.json`, `score.json`, `scenario.toml`, the container logs). `run.json` therefore also records when and why the run stopped and its last check, and each link in `score.json` its evidence.
- **Again, or for an earlier run:** `python3 -m vnapapi.report <results directory>` in `service/` writes the report for the results in `data/results/<run id>`. A run from before this feature has no stop time in `run.json`; the last log line stands in for it, and the links have no evidence column.
- **If it fails**, the other results are kept and `report-error.txt` holds the reason.

Suggested next runs

Section 6 of the report comes from the catalogue `service/recommendations.toml`. Each rule is a fixed suggestion with conditions on the run's facts; the report prints the rules whose conditions all hold, highest priority first, up to `limit` (6). The file is read each time a report is written, so a change needs no restart.

```
[[rule]]
id = "pattern-reversible"
priority = 90
when = ["links.wrong>=8", "pattern.share>=0.8"]
title = "Break the traffic pattern."
text = "{pattern.explained} of the {links.wrong} wrong links ({pattern.share:.0%}) follow one fixed rule
    ..."
```

Key	Meaning
`id`	a name of your own
`priority`	higher is printed first
`when`	conditions `fact OP value`, as in a scenario's `expect`: `OP` is one of `>= <= == != > <`, `value` a number or another fact. All must hold
`title`, `text`	the suggestion. `{fact}` inserts a fact; `{fact:.0%}` and `{fact:.1f}` format it. State the number that triggered the rule

The shipped rules, by priority:

Rule	Printed when	Suggests
`no-changes`	no vehicle changed its pseudonym	check the trigger
`pattern-reversible`	8 or more wrong links, 80 % of them explained by one fixed pairing	random turns or uneven arrivals
`starved`	changes refused for lack of certificates	a larger `refill_at` or `batch`
`timing-leak`	5 or more timing links, half or more correct and above chance	an image that rephases the CAM timer
`followed`	a vehicle followed through 3 or more changes in a row	a longer silent period, or a mix zone with `min_vehicles`
`above-chance`	8 or more links, correct twice as often as guessing	more vehicles changing together, longer silence
`zone-threshold`	more mix zone passages without a change than with	a lower `min_vehicles`, or vehicles arriving together
`pool-wrap`	a certificate was used again	a larger pool, or refill
`no-eavesdropper`	changes, and no eavesdropper	an `[eavesdropper]` section

Rule	Printed when	Suggests
`no-silent-period`	changes without a silent period	a silent period
`window-too-short`	the silent period can exceed the eavesdropper's link window	a longer `link_window`
`one-technique`	the eavesdropper links by position or timing only	both
`silent-cost`	vehicles silent 30 % of the run or more	a sweep of the silent period
`repeat`	fewer than 10 kept runs of this scenario	more runs with other seeds
`few-vehicles`	2 to 5 vehicles	six to ten
`equal-speeds`	all vehicles at one speed	different speeds

Facts (all numbers; 0 where the run has no eavesdropper):

Facts	Meaning
`duration_s`, `stations`, `vehicles`, `warnings`	length of the run, stations, stations with pseudonyms, warnings at stop
`changes`, `changes.rejected`, `changes.wrapped`, `changes.per_vehicle_min`	pseudonym changes; `wrapped`: back to a certificate used before
`silent.periods`, `silent.mean_s`, `silent.max_s`, `silent.share`, `silent.configured_max_s`	measured silent periods; `share` is the part of the vehicles' time (0 to 1)
`zone.count`, `zone.changes`, `zone.skipped`, `zone.min_vehicles`	mix zones: entries with a change event, passages without, largest threshold
`dynamic`, `dynamic.changes`	dynamic mix zone on (1), and its change events
`pki`, `pki.batches`, `pki.starved`, `pool.min`	the run's PKI, refill batches, changes refused for lack of certificates, smallest initial pool
`speeds.distinct`, `random_turns`	different speeds among the vehicles; vehicles with random turns
`repeats`, `repeats.seeds`	kept runs of the same owner with exactly this scenario text (this one included), and their different seeds
`eavesdropper`, `eavesdropper.link_window_s`, `eavesdropper.techniques`	present (1), its link window, and how many of position and timing it uses
`chance`	1 / vehicles: how often a guess is right
`links`, `links.correct`, `links.wrong`, `links.correct_share`, `links.over_chance`	the score; `over_chance` is `correct_share` / `chance`
`identifier.*`, `position.*`, `timing.*`	per technique: `links`, `correct`, `correct_share`
`pattern.explained`, `pattern.share`, `pattern.pairings`	wrong links one fixed pairing explains, as a share of the wrong links, and the number of pairings
`followed.longest`, `tracks`, `tracks.mixed`	longest correct chain; vehicle tracks, and those joining several vehicles

- **Checking a rule:** `python3 -m vnapapi.report --rules data/results/<run id>` in `service/` lists the run's facts and, per rule, `holds`, `no` with the conditions that failed, or `NOT USED` with the mistake (an unknown fact, a condition that cannot be parsed). It writes nothing, and exits with 1 if a rule is not used.
- **A rule with a mistake is never printed;** the report is still written.
- **Without the file** the report has no section 6.
- **Thresholds are starting values,** taken from the summaries in `results/`. One run is thin evidence, so each suggestion states its numbers.

Operations

vnapctl

docs/operations/vnapctl.md

`sim/vnapctl` starts, inspects and stops simulations described by scenario files. It needs only `python3`, the Docker CLI and the `eclipse-mosquitto:2` image. All commands below run in `${VNAP_HOME}/sim`.

Walkthrough

1. Choose a scenario. Each file in `scenarios/` describes the stations, their certificates, and optionally a control channel, movement, the run's own PKI and an eavesdropper. There are five foundational scenarios, one per phenomenon. Each has named **variants**: sets of overrides that switch one thing on or off, started as `<scenario>:<variant>`.

```
cd ${VNAP_HOME}/sim
./vnapctl scenarios          # name, stations, features and variants of every scenario
```

Scenario	Phenomenon	Measures	Variants
`trust`	signed CAMs accepted only when the signer's certificate chains AT → AA → trusted root (committed test certificates)	`chain.fail`, `cam.success_rate`	`badroot` (negative control), `v2`
`linkability`	what a pseudonym change leaks when the vehicle is alone: identifiers, CAM timing, position	links by technique, tracks	`certificate-only`, `silent`, `silent-patient`, `identifiers-only`, `manual`
`crossing`	privacy from other vehicles: four cars change together in a mix zone and leave in different directions	correct and wrong links, tracks	`min2`, `min3`, `blocks`, `blocks-min3`, `periodic`, `no-silence`
`convoy`	cars that change together but never separate: order and spacing survive the change	correct and wrong links, followed changes	`periodic`, `min2`, `no-silence`
`refill`	certificate supply from the run's PKI: refill latency, queueing, starvation	`pki.refresh_ms_*`, `pki.queue_ms_max`, `pki.starved`	`slow`, `batch4`, `batch16`, `batch32`, `pki-derives`, `no-refill`

The `controls/` folder holds pass/fail checks that measure nothing: `controls/none` (the link and CAM path without security), `controls/naive-v3` (self-signed certificates: every CAM rejected) and `controls/stress-naive-v3` (4 stations at 50 Hz, a regression test of the thread-safety fixes). `templates/` links the four foundational scenarios users of the service may start (all but `trust`, which uses certificate files). Where the scenarios before 2026-10-18 went is listed in scenario names.

2. Start it. `--dry-run` validates and prints the Docker commands. `up` returns when every station exchanges messages.

```
./vnapctl up trust --dry-run
./vnapctl up trust
```

3. Check it. `status` shows the stations, their certificates and chain checks. `check` observes for 15 s and gives a PASS/FAIL verdict.

```
./vnapctl status
./vnapctl check
```

4. Watch it. `./vnapctl events --duration 10s` merges received and sent messages with their verification results, and the pseudonym, ID change, PKI and certificate-check log lines. Raw frames and MQTT are covered in monitoring.

5. Change pseudonyms. In a pseudonym scenario the event client sends change events periodically; you can also send one by hand.

```
./vnapctl down && ./vnapctl up linkability
docker exec pseudo-client change 2          # station 2: next pseudonym
./vnapctl events --duration 10s --kind pseudonym,control
```

The client's commands and the event format are in control channels.

6. Play the attacker.

```
./vnapctl down && ./vnapctl up crossing
./vnapctl status                    # includes the eavesdropper's tracks and linked changes
docker logs -f eavesdropper
```

See eavesdropper and scoring.

7. Run a negative control. In `trust:badroot` the OBU trusts the wrong root and must reject the RSU's CAMs. The variant sets its own expectations, so `check` passes when the rejection happens.

```
./vnapctl down && ./vnapctl up trust:badroot && ./vnapctl check
```

8. Tear down.

```
./vnapctl down          # containers, networks, and the run's PKI volume with its keys
```

Commands

```
./vnapctl scenarios [--json]                          # list scenario files
./vnapctl validate my.toml [--json] [--as-user]       # field-level errors; --as-user applies the user
    policy
./vnapctl schema                                      # JSON Schema of the scenario format
./vnapctl up NAME[:VARIANT]|FILE [--set KEY=VALUE ...] [--dry-run] [--wait 30] [--instance N|auto]
    [--as-user]
./vnapctl status [--json]
./vnapctl events [--duration 10s] [--since 10m] [--kind K,...] [--station S] [--count N] [--json]
./vnapctl check [--duration 15s] [--expect EXPR ...] [--no-defaults] [--no-scenario] [--json]
./vnapctl down [--force] [--dry-run]
./vnapctl certs [status] [v2|v3] [--json]           # the committed test certificates' validity
./vnapctl certs regenerate v2|v3 [--lifetime KEY=N ...]   # a new set; see docs/certificates.md
```

Every command finishes on its own (none waits for Ctrl-C). Text output by default; `--json` for scripts and agents. `VNAPCTL_DEBUG=1` prints timings to stderr.

While a command works, it reports each step on stderr in lines starting with `vnapctl:` (e.g. building a helper image on first use, starting the run's PKI, waiting for the stations). The result stays on stdout. `-q`/`--quiet` and `--json` turn these messages off.

up and down

- **Validation:** `up` reports every problem at once before starting anything:
  - unknown keys and invalid security modes;
  - duplicate station names, IPs, IDs or MACs;
  - addresses outside their subnet;
  - missing certificate files;
  - certificate combinations the entrypoint cannot use.
- **Overrides:** `--set` changes any value without editing the file, e.g. `--set control.client.interval=10 --set defaults.security=certs-v2`:
  - an entry of an array of tables by its name or its index from 0: `--set stations.obu1.color="#ff0000"`, `--set control.mix_zones.crossing.min_vehicles=3`;
  - a whole value, arrays included, as TOML: `--set control.mix_zones=[]`;
  - every pseudonym station at once through `defaults.pseudonyms`: `--set defaults.pseudonyms.silent_max_ms=0` (a station's own key wins).

  Overrides are recorded in the container labels, so `status` and `check` see them.
- **Variants:** `up crossing:min3` applies the overrides of the scenario's `[variants.min3]` first, then any `--set`. The run is named `crossing:min3`, and the variant is recorded in the labels.

- **Refusals:** `up` does not start when a simulation already runs on the network, a container name is in use, a network exists with another subnet, or less than 0.5 GB of disk is free.
- **Rollback:** if a step fails, `up` removes whatever it created.
- **Helper images:** `up` builds the images a scenario needs (control client, mobility client, eavesdropper, PKI) when their tag is missing (build).
- **Readiness:** `up` returns when all of these hold (default wait 30 s, `--wait`):
  - every station has published on its MQTT broker;
  - pseudonym stations have joined the control channel;
  - moving stations have received their first position.

  Health, such as the expected chain failures of a negative control, is reported by `status` and `check`.
- **Cleanup:** `down` removes the containers with their anonymous volumes, the networks and the run's PKI volume. Bind mounts such as `certs/` are not touched.
- **Ownership:** containers are labelled with the scenario file, overrides, owner, start time and run ID. The owner is the account running `vnapctl`. `down` refuses runs started by another user, and unlabelled ones, unless given `--force`.
- **Secrets:** broker credentials are named by environment variable (`control.auth`). They reach the containers through `docker -e NAME`, never through the command line or labels.

Parallel instances

`--instance N` (before or after the subcommand) runs or inspects a copy on networks `vanetzalan0-iN` / `vnapctl0-iN`, with subnets 10.N.98.0/24 / 10.N.99.0/24 and container names `<name>-iN`. `status`, `events`, `check` and `down` take the same option.

- `up --instance auto` claims the lowest free N ≥ 1 by creating its simulation network. Docker refuses a duplicate name, so concurrent starts never share an instance.
- `up` prints the number it claimed; a failed start releases it.
- `VNAPCTL_INSTANCE` sets the default instance for every command.

Events

Each event has a timestamp, a station and a kind:

Kind	Source	Content
`rx`	`vanetza/out/<type>`	message type, sender, `secured`, `report`, size
`tx`	`vanetza/own/cam`	message type
`control`	`vnap/pseudonym/#`	change events and status answers
`pseudonym`	`[PSEUDONYM]` log lines	pool start, changes, rejections, control-channel connection
`idchange`	`[IDCHANGE]` log lines	GN address/MAC and `stationId` changes, ID-LOCK, silent periods
`pki`	`[PKI]` log lines	batch requests, installed batches, retries; issued batches
`chain`	`[V2-CHAIN]` / `[V3-CHAIN]` log lines	certificate checks: signature chain to the root CA, at startup and for refilled ATs
`error`	log lines	`Exit:`, failed assertions, crashes

- **Live and history:** MQTT has no history, so live capture covers `--duration`; `--since` adds older events from the container logs only.
- **Start-up gap:** the first ~1 s of a live window is missed while the subscriber containers start.

Checks

`check` collects events for `--duration`, computes metrics and evaluates expectations of the form `metric OP number|metric` (`>=`, `<=`, `==`, `!=`, `>`, `<`).

```
./vnapctl check --duration 20s --expect 'obu.pseudonym.changed>=1' --expect 'rsu.cam.rx_per_s>=0.9'
./vnapctl check --no-defaults --expect 'obu.cam.success_rate==0'     # negative controls
```

- **Station metrics** (per message type):
  - `<station>.<type>.rx`, `.rx_from.<station>`, `.success`, `.failed`, `.failed.<reason>`, `.success_rate`, `.rx_per_s`;
  - `<station>.<type>.tx`;
  - `<station>.pseudonym.changed`, `.rejected`, `.control_connected`;
  - `<station>.chain.fail`, `<station>.errors.total`;
  - refill metrics: see certificate refill.
- **Global metrics:** `control.event`, `control.status`, `control.changed`, `stations.running`, `stations.total`, `disk.free_gb`, `pki.*`, `eavesdropper.*`.
- **Metric list:** `check --json` returns every metric. A missing counter (e.g. no DENMs) counts as 0; a misspelled name fails and lists the metrics that exist.
- **Scenario expectations:** `check` also evaluates the running scenario's `[check] expect` list. With `[check] defaults = false`, as in the negative controls, the defaults are skipped. `--no-scenario` ignores the scenario.
- **Default expectations** (unless `--no-defaults`):
  - all stations running;
  - at least 2 GB free disk;
  - no failed chain checks and no crash lines;
  - every station receives CAMs from every other;
  - with security enabled, a CAM success rate of 1;
  - pseudonym stations connected to their control channel;
  - with refill, no starved changes and the run's PKI running.

Exit codes (all commands):

Code	Meaning
0	ok / pass
1	check failed
2	usage error or invalid scenario
3	nothing running
4	`status` degraded, or `up` not ready
5	conflict: already running, name in use, or owned by someone else

Your own scenarios

Copy a file in `scenarios/` and edit it. The format, the assigned fields and the user policy are in scenario format:

```
./vnapctl validate my.toml --json           # {"valid": false, "errors": [{"path": "stations[obu1].ip",
    ...}]}
./vnapctl up my.toml --instance auto
```

Scenario names before 2026-10-18

The catalogue was reduced to five foundational scenarios with variants. The earlier files are in the git history; each maps to a scenario and variant as follows.

Before	Now	Difference
`c-its-pki`	`trust`	none
`c-its-pki-badroot`	`trust:badroot`	none
`certify-fresh`	`trust:v2`	none
`c-its-pki-pseudo`, `c-its-pki-pseudo-manual`	`linkability`, `linkability:manual`	the OBU moves; certificates from the run's PKI
`c-its-pki-tracking`, `c-its-pki-tracking-silent`	`linkability`, `linkability:silent`	the OBU moves; certificates from the run's PKI
`c-its-pki-traffic`	`crossing:periodic`	four cars on the crossing instead of two on straight roads
`c-its-pki-mixzone`, `c-its-pki-mixzone-min3`	`crossing:blocks`, `crossing:blocks-min3`	certificates from the run's PKI
`c-its-pki-mixzone-random`, `c-its-pki-mixzone-pki`, `templates/mixzone-random`	`crossing`	`mixzone-random` used committed certificates
`c-its-pki-convoy`, `c-its-pki-convoy-dynamic`	`convoy:periodic`, `convoy`	certificates from the run's PKI, so no pool is reused
`c-its-pki-refill-sweep`	`refill` (`batch4` … `batch32`)	none
`c-its-pki-refill`, `templates/pki-refill`	`refill:slow`	four OBUs instead of two
`none`, `naive-v3`, `stress-naive-v3`	`controls/none`, `controls/naive-v3`, `controls/stress-naive-v3`	none

Service deployment

docs/operations/service-deployment.md

`service/` is the multi-user simulation service: an HTTP API (`vnapapi/`, FastAPI and SQLite) and a web UI (`ui/`) served by the same process. It drives simulations through the same `vnapsim` package as `vnapctl`. Users reach it only over HTTPS through a reverse proxy; the service account is the only one with access to Docker.

Quick start (development)

```
cd ${VNAP_HOME}/service
python3 -m vnapapi.admin create-user root --role admin   # first admin (password asked, or
    VNAP_NEW_PASSWORD)
./run.sh                                                 # 0.0.0.0:8080: web UI at /ui/, OpenAPI at
    /api/docs
```

- **Dependencies:** `run.sh` installs `requirements.txt` into `service/.venv` (or `.deps` without python3-venv) and installs again whenever `requirements.txt` changes.
- **Listen address:** all interfaces (`0.0.0.0`) by default. This is required when port 8080 is forwarded from the VM to the host, e.g. to open the UI at `http://localhost:8080` on a Mac. `VNAP_API_HOST=127.0.0.1` listens on the VM only; `VNAP_API_PORT` changes the port.
- **Plain HTTP:** the repository's `config.toml` sets `cookie_secure = false`, **for testing only**: the session cookie is then also sent over plain HTTP, so login works in every browser, including Safari and DuckDuckGo. With `cookie_secure = true` the browser keeps the cookie only over HTTPS or on `http://localhost` (Safari and other WebKit browsers drop it even there), and login over plain HTTP from another host is refused with an explanation. A production deployment must set `true` (see Production). For HTTPS in a development VM, see HTTPS from the host.

Production

On the VM:

```
sudo useradd --system --create-home --groups docker vnap
sudo -u vnap git clone --recurse-submodules <vnap-secure URL> /home/vnap/vnap-secure
sudo -u vnap make -C /home/vnap/vnap-secure service-deps   # service/.venv with the packages of
    requirements.txt
sudo -u vnap sed -i 's/^cookie_secure = false/cookie_secure = true/'
    /home/vnap/vnap-secure/service/config.toml   # production: HTTPS-only session cookie
sudo cp /home/vnap/vnap-secure/service/deploy/vnap-api.service /etc/systemd/system/
sudo cp /home/vnap/vnap-secure/service/deploy/vnap-backup.{service,timer} /etc/systemd/system/
sudo systemctl enable --now vnap-api vnap-backup.timer
sudo -u vnap sh -c 'cd /home/vnap/vnap-secure/service && .venv/bin/python -m vnapapi.admin create-user
    root --role admin'
```

- **Virtual environment:** `make service-deps` creates `service/.venv` and installs `requirements.txt` into it, as `vnap`, before the service first starts. It needs `make` and `python3.12-venv` (see installation). The `create-user` step uses `.venv/bin/python`, so the packages must be in place by then; `run.sh` would also install them on first start, but only while the service is starting.
- `cookie_secure = true`**:** the repository's `service/config.toml` has `false`, for testing only. The `sed` step sets `cookie_secure = true` before the service first starts, so the session cookie is never sent over plain HTTP.
- **Service account:** `vnap-api.service` runs `service/run.sh` as `vnap`, the only member of the docker group, with `NoNewPrivileges`, `ProtectSystem=full` and `UMask=0077`.
- **TLS:** `deploy/Caddyfile` terminates TLS (automatic certificates) and forwards to `127.0.0.1:8080`. Open only 443, and 80 for the certificate challenge.
- **Port 8080:** the service listens on all interfaces by default, so in production either block 8080 from outside (firewall), or bind it to localhost. For the latter, add

`Environment=VNAP_API_HOST=127.0.0.1` to `vnap-api.service`, so users reach it only through Caddy.

- **Test certificates:** regenerating them from the administration page writes `certs/` in the service's clone, as `vnap`, and runs `vnap-pki` and `vnap:latest` containers. Commit the new files from that clone (certificates).
- **Station images:** the station image must exist on the VM (`make image` as `vnap`, or `docker load`). Users may choose only the images listed in `sim/policy.toml`.
- **Upgrade:**

```
sudo -u vnap sh -c 'cd /home/vnap/vnap-secure && git pull && git submodule update --init'
systemctl restart vnap-api
```

Dependencies reinstall on start when `requirements.txt` changed, and the database schema is upgraded in place.

HTTPS from the host

The service speaks plain HTTP only. To reach it over HTTPS from the host of a development VM (e.g. because Safari drops the session cookie on `http://localhost`), put Caddy in front of it in the VM, with a certificate from Caddy's own local CA:

```
sudo apt install -y caddy
sudo cp /home/vnap/vnap-secure/service/deploy/Caddyfile.local /etc/caddy/Caddyfile
sudo systemctl restart caddy
```

1. **Forward a port** from the host to the VM's 8443, e.g. host port 48443, in the same way as the forward to 8080.
2. **Set** `cookie_secure = true` in `service/config.toml` (the repository has `false`, for testing only), then `sudo systemctl restart vnap-api`.
3. **Open** `https://localhost:48443` on the host. The certificate is issued for `localhost`, so the name matches whatever the host port is.

- **Certificate warning:** the browser does not know Caddy's local CA. Accept the warning, or trust the CA on the host. Copy `/var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt` from the VM, then on a Mac:

```
sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain root.crt
```

- **"Invalid HTTP request received"** in the service's log means a browser spoke HTTPS to the plain HTTP port (`https://` on the forward to 8080). Use `http://` there, or the HTTPS port.
- **Only through Caddy:** add `Environment=VNAP_API_HOST=127.0.0.1` to `vnap-api.service` and remove the forward to 8080.
- **Not for the internet:** this certificate is trusted only where the local CA is installed. A public deployment uses `deploy/Caddyfile` with a host name, as in Production.

Configuration

`service/config.toml` (or the file named by `VNAP_API_CONFIG`). Every key is described in configuration:

- the data directory and session lifetime;
- lockout and the login rate limit;
- TOTP;
- backups;
- per-role limits;
- container CPU and memory;
- map tiles;

- worker threads.

Accounts and authentication

- **Accounts:** built-in usernames and passwords, hashed with Argon2id (at least 12 characters). Roles: `admin`, `user`, `viewer`. Admins create accounts and reset passwords (web UI, or `python3 -m vnapapi.admin` on the server); there is no self-registration.
- **Sessions:** HttpOnly, Secure, SameSite=Strict cookies. Every state-changing request needs the session's `X-CSRF-Token` (returned by login and `/api/auth/me`).
- **API tokens** (`Authorization: Bearer vnap_...`): for automation, scoped to one user, revocable. Sessions and tokens are stored only as SHA-256 digests.
- **Lockout:** repeated failed logins lock the account for a while; login attempts are rate-limited per client address.
- **TOTP second factor** (optional, per user; RFC 6238: 6 digits, 30 s, SHA-1, as authenticator apps expect):
  - **Turning it on:** a user does this on the Account page: password, then scan the QR code (or type the key) and confirm with a code. Ten one-time recovery codes are shown once; new ones can be made later.
  - **Logging in** then needs the password and a code. A wrong code counts as a failed login, and a code is accepted only once.
  - **Turning it off** needs the password and a code. For a lost phone without recovery codes, an admin resets it (Admin page, or `python3 -m vnapapi.admin reset-totp <name>`); the user's sessions end and the password alone works again.
  - **Where it can be managed:** only from a login session. API tokens are not affected by TOTP.
  - **Storage:** the secrets are encrypted with AES-256-GCM under `[auth] totp_key_file`, which is outside the data directory. See backup and restore.
- **Audit:** security-relevant actions go to the audit log (`/api/audit`, Admin page):
  - logins, failures and second factors;
  - run start and stop, control actions;
  - account, token and TOTP changes;
  - backups.

`python3 -m vnapapi.admin` on the server: `create-user`, `reset-password`, `unlock`, `reset-totp`, `list-users`. Passwords come from the terminal or `VNAP_NEW_PASSWORD`, never from the command line.

Runs

- **What users start:** templates (`sim/scenarios/templates/`) or their own scenario text, with overrides. All are checked against the user policy, with field-level errors. Admins can also start the catalogue (`sim/scenarios/`).
- **Saved templates:** a user saves scenario text as a template of their own, optionally shared with all users (at most 100 per user). They are in the database, so backups include them; the service's own templates stay in `sim/scenarios/templates/`.
- **Jobs:** starting and stopping are jobs. Each run gets:
  - a free instance;
  - CPU and memory limits on every container;
  - broker credentials generated by the service, which users never see.
- **Limits:** per role, a number of concurrent runs, a longest duration (runs stop automatically at their deadline) and disk space for kept results.

- **Identifier:** `<user>-<scenario name>-<n>`, e.g. `alice-crossing-3` (`alice-crossing-min3-1` for a variant): the owner, the name the run was started with (its scenario's, unless another was given) and a counter, which starts at 1 for each user and name. A number is never given twice, also after a run was deleted. Characters other than letters, digits, `.`, `_` and `-` become `-`; a scenario text without any name gives `<user>-run-<n>`. Renaming a run later does not change its identifier. Runs from before this scheme keep their 12 hex digits.
- **Record:** a run keeps its scenario, overrides, image, mobility seed and the C-ITS-PKI commit of its PKI.
- **Name and description:** a run starts with its scenario's; the owner can change both at any time (web UI, or `PATCH /api/runs/{id}`). `run.json` in the results follows.
- **Privacy:** runs are private to their owner (and admins) unless shared with other users, read-only.
- **Results:** at stop, the service keeps:
  - the containers' logs and the eavesdropper's logs;
  - the status;
  - the scoring (`score.json`; see eavesdropper and scoring);
  - the PKI's issue log (digests only, never keys);
  - `run.json` and the scenario;
  - a summary report (`report.docx`), written from the files above. See summary report.

Tests

```
make test-service      # 36 tests, no Docker needed
```

The endpoints are listed in API; the browser UI is described in web UI.

Web UI

docs/operations/web-ui.md

The service serves a browser UI at `/ui/` (`/` redirects there). With it, a user can author, run, observe and evaluate a scenario without the command line. It is plain JavaScript modules (`service/ui/`), with no build step and no third-party code except Leaflet (vendored in `service/ui/vendor/leaflet/`), and it uses only the API.

Pages

- **Runs:** every visible run with its state and time left; stop and delete.
- **New run:**
  1. *Start from the template*: pick a template (or the blank scenario). Its text becomes the new scenario. The list has the service's templates (the foundational scenarios `linkability`, `crossing`, `convoy` and `refill`), the saved templates and, for admins, the catalogue (also `trust` and the controls, as `controls/<name>`). Picking another one replaces the scenario, after a question if it was edited.
  2. Edit the scenario in either tab; both edit the same scenario:
     - *Scenario text*: TOML (scenario format).
     - *Builder*: RSUs and vehicles, pseudonyms and silent periods, routes or random-turn crossings, mix zones, the run's PKI (certificates at start, refill) and the eavesdropper. Positions, routes, crossings and zones are set by clicking the map. Each vehicle has a colour, used for the vehicle and its route here and on the run's map. *Dynamic mix zone*: "all vehicles" with "min cars", radius and cooldown for the run, and a toggle with its own "min cars" per vehicle (vehicle movement).
  3. Pick a variant of the scenario if it has any (its overrides and description are shown), add overrides, validate, start. The variant's overrides apply first, then the ones typed here (variants). The scenario is checked against the user policy. Field errors are listed, and in the builder the station concerned is highlighted.
  - **Text and builder:** the builder reads the text when its tab opens, and keeps every key its forms do not show (e.g. `[check]`, `validity_hours`). It writes the text again only when something was changed in it; the text's comments are then lost, and defaults the forms show are written out (e.g. a station's position).
  - **Save as a template:** saves the scenario as it is, under a name (letters, digits, `-`, `_`); it must be valid. A saved template is its owner's; "shared with all users" lets every user start from it. Only the owner or an admin replaces or deletes it.
  - **Generic names:** when saving, renames the stations `rsu1`, `rsu2`, ... (station type 15) and `vehicle1`, `vehicle2`, ..., and the mix zones `zone1`, `zone2`, ..., also where `[check]` expectations start with a station's name. Comments keep the old names.
- **Run:**
  - *Overview*: the run's name and description, scenario, owner, instance, time left, image and seed. Per station: state, pseudonym, refill and certificate checks. Also the run PKI's issue times, the eavesdropper's tracks and warnings. Stop and share are here. "Edit name and description" changes both, in any state of the run (owner or admin). They start as the scenario's; an empty name gives the scenario's name again.
  - *Map*: start positions, routes, crossings and mix zones, with live positions every 2 s. Pick a station, then click the map to move it.
  - *Events*: the last 10 minutes' events and then the live event stream, filtered by kind (pseudonym, ID change, refill, certificate check, error), with pause. Hover a kind for its meaning.

  - *Control*: ID change trigger, lock and unlock, with each station's answer.
  - *Results*: run a check (extra expectations allowed) and see earlier checks. After the stop, download the summary report (`report.docx`) and the other collected files, and see the eavesdropper's score against ground truth; the headings explain each column on hover.
- **Account:**
  - change password;
  - turn the TOTP second factor on (QR code) or off, and make new recovery codes;
  - create and revoke API tokens (a new token is shown once).
- **Admin:**
  - users: create, change roles, disable, unlock, reset passwords, reset a user's TOTP;
  - backups: list them, and back up now;
  - test certificates: every certificate of the committed v2 and v3 sets with its validity (expired ones marked), and *Regenerate* with a lifetime per certificate kind. The fields start at the lifetimes of the last regeneration, *Defaults* restores the tools' defaults, and the page follows the job until it is done (certificates);
  - the audit log, filtered by user.

Reading the eavesdropper score

The *Results* tab compares what the eavesdropper concluded with what really happened (the stations' own logs):

- **Links:** pseudonym changes the eavesdropper linked; correct when both pseudonyms belong to the same vehicle.
- **Longest chain followed:** the most pseudonym changes through which it followed one vehicle without a mistake. This is the privacy result in one number; 0 means it never followed a vehicle through a change.
- **Per track:**
  - *Identities*: pseudonyms in the track.
  - *Most Frequent*: the share that belongs to its most frequent vehicle, with that vehicle in parentheses, e.g. `75 % (obu1)`. Several names mean a tie.
  - *Followed*: changes followed without a mistake.
  - *Station predictions in actual order across identity changes*: the real vehicle behind each pseudonym the eavesdropper put into the track, in the order it added them, e.g. `obu1 → obu1 → obu2`. One name throughout means it followed that vehicle; a change of name is a wrong link.

  Read the share together with identities. 1 identity is always 100 % and means nothing was linked. Many identities at a high share means that vehicle was tracked. A low share means different vehicles were mixed up, so the mix zone worked.

Details are in eavesdropper and scoring.

Browser notes

- **Session cookie:** HTTPS-only with `cookie_secure = true`, as production must have it. Safari and other WebKit browsers (including DuckDuckGo) then drop it even on `http://localhost`; use HTTPS, Chrome or Firefox. The repository's `config.toml` sets `cookie_secure = false`, for testing only, so that these browsers work over plain HTTP. The UI says so when the cookie was dropped.
- **Map tiles** come from `[ui] tile_url` (OpenStreetMap by default) and are requested with an origin-only Referer, as the OpenStreetMap tile policy requires. Without internet access, set an internal tile server, or `""` for no background map.

Security

- **Content-Security-Policy:** strict. Scripts and styles come only from the service, with no inline code; images may also come from the tile server.
- **Framing and referrers:** `frame-ancestors 'none'`; no referrer, except the origin for map tiles.
- **Text** is only ever inserted as text, never as HTML.
- **Sessions:** the session cookie and CSRF token are the API's.

Backup and restore

docs/operations/backup-restore.md

What is backed up

An archive holds the service's data directory:

- **Database:** a consistent online snapshot of the users, runs, checks and audit log.
- **Results:** every run's collected logs, scores and issue logs.
- **User scenarios:** the users' scenario texts.

Not in a backup:

- **The TOTP key** (`[auth] totp_key_file`): see below.
- **The runs' PKI keys:** they exist only while a run does.
- **Running simulations:** containers and networks.

An archive contains password hashes and the audit log. Copy it off the VM only encrypted (for example with `age`, `gpg`, or an encrypted rclone remote).

Commands

```
cd ${VNAP_HOME}/service
./backup.sh create               # new archive in [backup] dir; the oldest beyond [backup] keep are
    deleted
./backup.sh list                 # archives, newest first
./backup.sh verify <archive>     # every file against the manifest, and the database's integrity
```

- **Archive:** `backups/vnap-backup-<UTC time>.tar.gz`, mode 0600, with a manifest of SHA-256 digests. `[backup] keep` (default 14) archives are kept.
- **Daily backups:** `deploy/vnap-backup.timer` runs `create` at 03:15 as the service account (and at the next boot if the VM was off then).
- **Admins** can also click *Back up now* on the Admin page, which lists the archives too. Archives are downloaded on the server, not through the API.

Restore

With the service stopped:

```
systemctl stop vnap-api
sudo -u vnap sh -c 'cd /home/vnap/vnap-secure/service && ./backup.sh restore <archive> --force'
systemctl start vnap-api
```

`restore`:

1. verifies the archive first;
2. keeps the current data directory as `data.before-restore-<time>`;
3. follows a change of data directory (e.g. a restore on another VM), so runs still find their scenario texts;
4. marks runs that were active at backup time as failed, since their containers are not in a backup.

Without `--force` it refuses to replace a data directory that is not empty.

The TOTP key

TOTP secrets must be stored reversibly, because the server computes the same codes as the user's app. They are encrypted with AES-256-GCM under the key in `[auth] totp_key_file` (default `service/secrets/totp.key`). The key is created on first use, mode 0600, outside the data directory, so the database or a backup alone does not reveal anyone's second factor.

- **Keep a copy** of the key file in a safe place, separately from the backups (once; it does not change).
- **If the key is lost,** every user's TOTP must be reset: Admin page, or `python3 -m vnapapi.admin reset-totp <name>`. Those users log in with their password and set TOTP up again.
- **Recovery codes** are stored as digests and do not depend on the key.

Monitoring

docs/operations/monitoring.md

`vnapctl status`, `events` and `check` (vnapctl) cover most needs. This page shows the underlying sources, for a closer look.

The examples use instance 0, with stations named as in the scenario (`rsu`, `obu`, ...) on `vanetzalan0` (192.168.98.0/24): the RSU usually at .10, the first OBU at .20. For instance N, add `-iN` to container and network names and use 10.N.98.x addresses. `vnapctl status` shows each station's address.

What	Where to look	Shows
Received messages	MQTT `vanetza/out/<type>` on the receiving station's broker	the decoded message as JSON, plus its verification result
Sent CAMs	MQTT `vanetza/own/cam` on the sending station's broker	the station's own CAMs as sent
Frames on the link	`tcpdump` on `br0` inside a station	raw GeoNetworking frames (ethertype 0x8947), including the security header
Pseudonym, position and PKI traffic	MQTT `vnap/#` on the control broker (`pseudo-broker`, network `vnapctl0`)	change events and answers, positions, certificate batches; see control channels
Station logs	`docker logs <station>`	`[V3-CHAIN]`, `[PSEUDONYM]`, `[IDCHANGE]`, `[PKI]`, `[MOBILITY]` lines

Application messages (MQTT)

Each station runs an embedded MQTT broker on port 1883. socktap publishes every message it receives on `vanetza/out/<type>` (`cam`, `denm`, `cpm`, ...):

```
# everything the RSU receives, with the topic
docker run --rm --network vanetzalan0 eclipse-mosquitto:2 \
  mosquitto_sub -h 192.168.98.10 -t 'vanetza/out/#' -v

# one line per received CAM: sender, receiver, verification result, payload size
docker run --rm --network vanetzalan0 eclipse-mosquitto:2 \
  mosquitto_sub -h 192.168.98.10 -t vanetza/out/cam |
  jq -c '{from: .stationID, to: .receiverID, secured, report: .security_report.description, size:
    .packet_size}'
#  {"from":2,"to":1,"secured":true,"report":"Success","size":82}

# only messages that failed verification
docker run --rm --network vanetzalan0 eclipse-mosquitto:2 \
  mosquitto_sub -h 192.168.98.10 -t 'vanetza/out/#' |
  jq -c 'select(.security_report.description != "Success")'

# CAMs the OBU sends
docker run --rm --network vanetzalan0 eclipse-mosquitto:2 \
  mosquitto_sub -h 192.168.98.20 -t vanetza/own/cam
```

Useful JSON fields:

- `stationID` / `stationAddr`: the sender's station ID and MAC address (both change with a full ID change).
- `receiverID` / `receiverType`: the receiving station.
- `secured`, `security_report.description`: the verification result, `Success` or a failure reason such as `Invalid_Certificate`.
- `packet_size`: size of the message payload in bytes.
- `fields`: the decoded message, e.g. `fields.cam.camParameters.basicContainer.referencePosition`.

socktap always delivers received messages, even those that fail verification (non-strict decapsulation). Judge a message by `security_report`, not by whether it arrived.

Frames on the link (tcpdump)

The station image includes `tcpdump`. The station's link interface is the bridge `br0` (`vnapctl` sets `SUPPORT_MAC_BLOCKING=true`):

```
# live, GeoNetworking frames only (both directions, as seen by the RSU)
docker exec rsu tcpdump -i br0 -e -nn ether proto 0x8947

# frames from one sender MAC only
docker exec rsu tcpdump -i br0 -e -nn ether src 6e:06:e0:03:00:02 and ether proto 0x8947

# save a capture for Wireshark (Ctrl-C to stop); under /mnt/hgfs it is visible on the host
docker exec rsu tcpdump -i br0 -U -w - ether proto 0x8947 > /mnt/hgfs/<host-directory>/vnap.pcap
```

- **tcpdump output:** it decodes the GeoNetworking and BTP headers. It does not parse the secured header, so for signed packets the values printed after it are wrong (e.g. `Payload:`, `lat`/`lon`).
- **Wireshark:** its GeoNetworking, BTP, ITS (CAM/DENM) and IEEE 1609.2 dissectors decode the full frame, including the signer certificate. Wireshark is not installed in the VM; open the capture on the host.
- **Frame size:** with v3 and 1 Hz CAMs, every CAM carries the full signer certificate, so frame size does not reveal a pseudonym change.
- **Offline analysis:** the eavesdropper decodes a capture offline (`python3 sim/images/eavesdropper/eavesdropper.py --pcap capture.pcap --log-dir out/`, needs `asn1tools`).

Control channel

```
# all control traffic (add -u USER -P PASSWORD if the broker requires them)
docker run --rm --network vnapctl0 eclipse-mosquitto:2 mosquitto_sub -h pseudo-broker -t 'vnap/#' -v

docker logs -f pseudo-client                # events sent by the client and the answers
docker logs -f mobility                     # vehicle positions every 10 s, laps, mix-zone events
docker logs -f pki                          # provisioning, issued batches with timings
```

For runs started by the service, the broker requires the run's credentials, which the service keeps to itself. Use the run's *Events* tab, or `/api/runs/<id>/events`, instead.

Troubleshooting

docs/operations/troubleshooting.md

First steps

- `./vnapctl status`: stations, their certificates and certificate checks, warnings (stopped containers, failed chain checks, unconnected control channels, starved refills, low disk).
- `docker logs <station>`: socktap's own output.
- `VNAPCTL_DEBUG=1 ./vnapctl …`: `vnapctl`'s timings.
- `./vnapctl check --json`: every metric, to see which expectation failed and why.

Startup certificate checks

Each station logs the result of checking its configured chain and own certificates (`docker logs rsu`, or `vnapctl events --kind chain --since 5m`):

```
[V3-CHAIN] chain certificate /vnap-certs/c-its-pki/aa.cert: OK
[V3-CHAIN] own authorization ticket /vnap-certs/c-its-pki/at.cert: OK
```

A wrong root, an expired certificate or a mismatched key shows up here. Refilled ATs are checked the same way (`batch authorization ticket …`). The checks are described in security and chain verification.

Received messages

Received messages are not logged to stdout. socktap publishes them on MQTT, and `security_report` holds the verification result (monitoring). Messages that fail verification are still delivered, with a failing `security_report`.

Checking message files offline

Signed or encrypted message files (e.g. from C-ITS-PKI) can be checked with Vanetza's own security code:

```
make msgcheck                                  # builds vnap:msgcheck (after make image - takes ~20
    minutes)
docker run --rm -v DIR:/w vnap:msgcheck v3 /w/msg /w/at.cert /w/aa.cert /w/root_ca.cert
docker run --rm -v DIR:/w vnap:msgcheck decode-v3 /w/msg.enc
```

`DIR` must be under the real `$HOME` (snap-confined Docker).

Common problems

Symptom	Cause and fix
`up`: "C-ITS-PKI not found … git submodule update --init"	the submodule is not initialised: `make submodule`
`make image`: "does not contain the base commit"	`VANETZA_NAP_DIR` holds another vanetza-nap history: check out the base there, or use an empty directory
Image build stops with "no space left on device"	the first build needs about 10 GB of build cache; grow the disk (installation) or remove unused images
Docker cannot read a build context or file on `/mnt/hgfs`	snap-confined Docker: use the provided scripts (they stream contexts on stdin) and keep mounts under `$HOME`
`up` refuses: "already running", "name in use"	another run uses the instance: `--instance auto`, or `down` the other run
`down` refuses: "started by …"	the run belongs to another account; `--force` if you are sure

Symptom	Cause and fix
Web UI: login succeeds, then 401	the browser dropped the HTTPS-only session cookie (web UI)
Web UI: grey map tiles saying "Referer is required"	an old UI version cached; reload with the cache cleared
A station's CAMs fail with `Invalid_Timestamp`	clock skew between containers; the patched image keeps its runtime clock synchronised. Check that the image is patched
`check`: `eavesdropper.*` metrics missing	the scenario has no `[eavesdropper]` section
`check`: stations send CAMs (`*.cam.tx`) but receive none, nothing in the logs (`trust` and other scenarios with certificate files)	the committed certificates expired; stations drop such messages silently. `./vnapctl certs` shows it; `./vnapctl certs regenerate v3 --lifetime at_hours=…` (or v2) makes new ones (certificates)

Known limitations

These come from upstream Vanetza-NAP:

- socktap always uses non-strict decapsulation, so messages that fail verification are still delivered, with a failing `security_report`.
- v2 puts `--certificate-chain` AAs into the cache without checking them against the trusted root.
- The v2 verifier accepts only payload type `signed`.
- ThreadSanitizer still reports races inside the Zenoh library's own threads (14 in a 3-minute c-its-pki run). They involve uninstrumented Rust code whose synchronization ThreadSanitizer cannot see, so they are most likely false positives. It reports none in socktap or Vanetza code.

And from the simulation itself:

- A pseudonym change (with the default full ID change) changes the certificate, GN address, MAC and CAM `stationId` together. It does not change the vehicle's broadcast position, so position continuity remains a linking clue (eavesdropper).
- The control channel uses one account per run and no TLS (control channels).

Reference

Scenario format

docs/reference/scenario-format.md

One TOML file per scenario in `sim/scenarios/`: the five foundational scenarios, pass/fail controls in `controls/`, and in `templates/` links to the scenarios the service's users may start (vnapctl). `vnapctl up <name>` starts one, `vnapctl up <name>:<variant>` one of its variants, `vnapctl scenarios` lists them, `vnapctl schema` prints the whole format as JSON Schema, and `vnapctl validate --json` reports errors by field.

Paths under `/vnap-certs/` refer to `certs_dir`: by default the repository's `certs/`; a relative `certs_dir` is resolved from the scenario file. It is mounted read-only into every station. Docker from snap can only mount directories under your home directory.

```
name = "example"                  # default: file name
description = "one line"
image = "vnap:latest"             # must exist locally
entrypoint = "native"             # "native": the image's patched entrypoint.sh
                                  # "stock": unpatched image, mounts sim/stock-entrypoint.sh
certs_dir = "/home/demo/pki-test-v2"   # optional

[network]                         # the V2X link
name = "vanetzalan0"
subnet = "192.168.98.0/24"

[defaults]                        # inherited by every station
security = "certs-v3"             # none | dummy[-v2|-v3] | certs-v2 | certs-v3
aa_cert = "/vnap-certs/c-its-pki/aa.cert"
root_cert = "/vnap-certs/c-its-pki/root_ca.cert"
env = { }                         # extra container environment
pseudonyms = { silent_min_ms = 3000, silent_max_ms = 13000 }   # optional: keys for every station that
                                  # has a pseudonyms table; the station's own keys win

# ip, station_id, mac and station_type are optional: left out, vnapctl assigns the next free
# station ID, address (.10, .20, ...), a MAC derived from the ID and type 5 (an RSU needs 15).
# Empty tables ([control], [eavesdropper], pseudonyms = {}) enable a feature with its defaults.
[[stations]]
name = "rsu"
ip = "192.168.98.10"
station_id = 1
station_type = 15
mac = "6e:06:e0:03:00:01"
at_cert = "/vnap-certs/c-its-pki/at.cert"     # static AT ...
at_key = "/vnap-certs/c-its-pki/at.der"

[[stations]]
name = "obu"
# ...
# ... or a pseudonym pool (TOML inline tables must stay on one line):
pseudonyms = { cert = "/vnap-certs/c-its-pki/bke_at_{i}.cert", key =
    "/vnap-certs/c-its-pki/bke_at_{i}_sign.der", count = 8, min_interval_ms = 1000, id_change = "full",
    silent_min_ms = 0, silent_max_ms = 0 }
#   id_change: "full" (ETSI ID change: GN address, MAC, stationId change too) or "certificate"
#   silent_min_ms/silent_max_ms: random radio silence after each full ID change (0 = off)
#   first: index of the first file ({i} = first .. first+count-1, default 0), so stations can
#          split one numbered set, e.g. first = 0, count = 4 and first = 4, count = 4
# a fixed position (default: 40.208106, -8.4197756, the crossing of Rua Alexandre Herculano and
# Rua Venancio Rodrigues in Coimbra, ORIGIN in sim/vnapsim/common.py; the example layouts are
# built around it):
env = { VANETZA_LATITUDE = "40.208106", VANETZA_LONGITUDE = "-8.4197756" }
# optional movement (needs [control]); the station starts at the first waypoint:
mobility = { route = [[40.208106, -8.4227848], [40.208106, -8.4167664]], speed_kmh = 50, start_s = 0,
    loop = true }
#   route: two or more [lat, lon] waypoints in degrees, driven at constant speed_kmh
#   start_s: seconds to wait at the first waypoint; loop: drive back to the first waypoint and
#   repeat (otherwise stop at the last one)
# ... or laps through a four-way crossing with random turns (instead of route and loop):
mobility = { crossing = [40.208106, -8.4197756], arm_m = 150, start_arm = "east", speed_kmh = 36, start_s
    = 0 }
#   arms of arm_m metres end on a square ring road; each lap: random turn at the crossing, random
#   way along the ring, back in on the next arm (4 x arm_m per lap)

[pki]                             # optional: the run's own certificate authority (needs [control])
initial = 8                       # butterfly ATs each pseudonym station starts with
refill_at = 2                     # request a new batch at this many unused ATs (0: no refill, pool wraps)
batch = 8                         # ATs per batch
validity_hours = 24
```

```
# etsi_version = "v3"             # default: from the stations' security (certs-v3 / certs-v2)
# ip = "192.168.99.5"             # PKI service on the control network
# key_derivation = "station"      # stations derive their ATs' private keys (default); "pki": the PKI
    returns them
# With [pki], stations take no certificate paths: pseudonym stations use
#   pseudonyms = { initial = 8, refill_at = 2, batch = 8, min_interval_ms = 1000, ... }
# (all optional), other stations with security certs-* get a regular AT.

[control]                         # optional control channel (pseudonym events, positions, refill)
network = "vnapctl0"
subnet = "192.168.99.0/24"
broker_ip = "192.168.99.2"
client_ip = "192.168.99.3"
mobility_ip = "192.168.99.4"      # mobility client, started when a station has mobility
mobility_rate_hz = 5              # position updates per second and vehicle
mobility_seed = 12345             # optional: repeat the random turns of a run (default: a new seed)
auth = { username_env = "CTL_USER", password_env = "CTL_PASS" }   # optional; names of env vars, never
    values

[control.dynamic_mix_zone]         # optional: a change when enough vehicles are near each other (vehicle
    movement)
enabled = true                    # run scope: every station with mobility and pseudonyms takes part
    (default true)
min_vehicles = 2                  # vehicles within the radius, the vehicle itself included (default 2)
radius_m = 50
cooldown_s = 30                   # least time between two such changes of a vehicle

[[control.mix_zones]]              # optional, repeatable; needs stations with mobility and pseudonyms
name = "crossing"
center = [40.208106, -8.4197756]  # [lat, lon]
radius_m = 40                     # pseudonym change event when a vehicle enters this circle
stations = [2, 3]                 # default: every station with mobility and pseudonyms
min_vehicles = 2                  # optional (default 1): change only once this many of them are inside

[control.client]
mode = "periodic"                 # periodic | random | once | manual
interval = 30                     # periodic; random uses min_interval/max_interval; once uses delay/index
stations = [2]                    # default: all stations with a pseudonym pool

[eavesdropper]                    # optional passive tracking attacker on the message network
ip = "192.168.98.99"              # default: host .99 of the network
summary_interval = 30             # seconds between console summaries
link_by = ["position", "timing"]  # evidence beyond shared identifiers (default both; [] = identifiers
    only)
link_window = 3.0                 # position-continuity linking: max silence before a new pseudonym
link_distance = 50.0              #   and max distance (m, plus speed x gap)
timing_window = 20.0              # timing-phase linking: max silence over which CAM timing is extended
timing_tolerance_ms = 25.0        #   and max difference between predicted and actual CAM time
verbose = false                   # log every frame to the console

[check]                           # used by "vnapctl check" while this scenario runs
defaults = true                   # include vnapctl's default expectations
expect = ["rsu.cam.rx_per_s>=0.8"]

[variants.silent]                 # optional, repeatable: a named set of overrides (see Variants)
description = "3-13 s silent period"
set = ["defaults.pseudonyms.silent_min_ms=3000", "defaults.pseudonyms.silent_max_ms=13000"]
```

Overrides

Any value can be overridden at start, without editing the file:

```
./vnapctl up trust --set defaults.security=certs-v2 --set certs_dir=/home/demo/pki-test-v2 \
    --set stations.obu.root_cert=/vnap-certs/c-its-pki/tlm.cert
```

- **Values** are TOML (`--set 'stations.obu2.pseudonyms={initial=4}'`); a whole array too (`--set control.mix_zones=[]` removes the mix zones).
- **Entries of arrays of tables** are addressed by name or by index from 0: `stations.obu2.color`, `control.mix_zones.crossing.min_vehicles`, `control.mix_zones.0.radius_m`.
- **Every pseudonym station at once:** `defaults.pseudonyms.<key>`. This reaches only stations that do not set the key themselves, so the foundational scenarios keep shared pseudonym settings in `[defaults]`.

Overrides are stored in the container labels, so `vnapctl status` and `check` see them; the service records them with the run.

Variants

A variant is a named set of overrides kept in the scenario file, so that the settings studied in the test summaries stay one name away:

```
[variants.min3]
description = "a car that entered changes only once 3 cars are inside the zone"
set = ["control.mix_zones.crossing.min_vehicles=3"]
```

- **Starting one:** `vnapctl up crossing:min3`; in the web UI, the Variant list under the scenario; in the API, `"variant": "min3"` next to the template or scenario.
- **Order:** the variant's overrides first, then `--set` or the run's overrides.
- **Recording:** the run is named `<scenario>:<variant>` (`<user>-crossing-min3-<n>` in the service). `vnapctl` records the variant in the container labels; the service stores the variant's overrides in front of the run's own, so a stored run starts and reloads without the scenario's variant table.
- **Checks:** `make test` loads every scenario with each of its variants, so a variant that no longer applies fails the tests.
- **User policy:** a variant's overrides are checked like any others: one that sets an administrator's field is refused for users.

Validation

- **Field-level errors:** every validation error has a field path (e.g. `stations[obu1].ip`), a message and the text `vnapctl` prints. `vnapctl validate --json` and the service return them, and the web UI highlights the field.
- **Checked:**
  - unknown keys and invalid values;
  - duplicate station names, IPs, IDs or MACs;
  - addresses outside their subnet;
  - missing certificate files;
  - certificate combinations the entrypoint cannot use;
  - certificate paths together with `[pki]`;
  - mix zones without moving pseudonym stations;
  - a dynamic mix zone threshold above the number of stations taking part.
- **Colour:** a station's `color = "#rrggbb"` is the colour of the station and its route on the web UI's maps. The simulation does not use it.
- **Dynamic mix zone, station scope:** a station's `dynamic_mix_zone = { enabled = false }` or `{ min_vehicles = 3 }` takes it out of the run scope or gives it its own threshold; with `enabled = true` it takes part even when the run scope is off (vehicle movement).
- **Assigned identities:** stations may leave out `ip`, `station_id`, `mac` and `station_type`. `vnapctl` assigns:
  - the next free station ID from 1;
  - the next free host (.10, .20, … .240, then the others; .1 and .99 are reserved);
  - a MAC derived from the station ID;
  - type 5 (passenger car). An RSU needs `station_type = 15`.

User policy

Users of the service (and `vnapctl validate|up --as-user`) are restricted by the user policy. Administrators, i.e. plain `vnapctl`, are not.

- **Admin fields:** the schema marks some fields `admin`: images, container environment, certificate files, networks, broker credentials. These are refused, except the images and environment variables `sim/policy.toml` allows:

```
approved_images = ["vnap:latest", "vnap:r2-p19"]   # scenario key "image"
allowed_env = ["VANETZA_LATITUDE", "VANETZA_LONGITUDE"]
```

- **Assigned fields:** fields the schema marks `assigned` (addresses, IDs) must be left out; the service fills them in.
- **Certificates:** user scenarios get their certificates from the run's own PKI (`[pki]`), or use none.

Configuration

docs/reference/configuration.md

Paths in this documentation

Variable	Meaning
INSTALL_DIR	the directory the repository is cloned into
VNAP_HOME	the repository: ${INSTALL_DIR}/vnap-secure

Both are set during installation, and the commands in this documentation use them to change directory. No script or program reads them: each finds the repository from its own location.

Service: service/config.toml

Read at start (VNAP_API_CONFIG names another file). Relative paths are relative to service/. Missing keys take the defaults shown.

Section	Key	Default	Meaning
[server]	data_dir	"data"	database, users' scenario texts, collected results (backed up)
	cookie_secure	true	session cookie only over HTTPS. The repository's config.toml sets false, **for testing only** (login over plain HTTP in every browser); production must set true
	session_hours	12	session lifetime
[auth]	max_failed_logins	5	failed logins (password or TOTP) before the account is locked
	lockout_minutes	15	lock duration
	login_attempts_per_minute	10	per client address
	totp_issuer	"vnap-secure"	name authenticator apps show
	totp_key_file	"secrets/totp.key"	AES-256 key for TOTP secrets; outside data_dir, keep a copy (backup and restore)
[backup]	dir	"backups"	where archives are written
	keep	14	archives kept (0: all)
[limits.user], [limits.admin]	concurrent_runs	2 / 10	runs at the same time (viewers start none)
	max_duration_minutes	60 / 480	longest run; stopped automatically at the deadline
	results_mb	500 / 5000	disk for kept results
[containers.station], [containers.pki], [containers.default]	cpus, memory	1.0/512m, 1.0/512m, 0.5/256m	limits per container role
[ui]	tile_url	OpenStreetMap	map tile URL template; "" for no background map
	tile_attribution	© OpenStreetMap contributors	shown on the map
[worker]	threads	2	runs starting or stopping at the same time

Environment:

- `VNAP_API_HOST`, `VNAP_API_PORT`: the listen address for `run.sh` (default `0.0.0.0:8080`, all interfaces). All interfaces are required for forwarding port 8080 from the VM to the host. `127.0.0.1` listens on the VM only, e.g. behind Caddy in production (service deployment).
- `VNAP_API_CONFIG`: another configuration file.
- `VNAP_SIM_DIR`: where `vnapsim` is (default `../sim`).
- `CITS_PKI_DIR`: another C-ITS-PKI (architecture).

Report suggestions: `service/recommendations.toml`

The rules behind "Suggested next runs" in a run's summary report: conditions on the run's facts, fixed text, priorities and a limit. Read for every report, so a change needs no restart. See suggested next runs.

User policy: `sim/policy.toml`

What a scenario from a user may set (scenario format):

Key	Meaning
`approved_images`	station images users may choose (`image`)
`allowed_env`	container environment variables users may set per station (`env`)

vnapctl

`vnapctl` has no configuration file; scenario files and options select everything. Its environment:

- `VNAPCTL_INSTANCE`: the default instance.
- `VNAPCTL_DEBUG=1`: timings on stderr.
- `CITS_PKI_DIR`: another C-ITS-PKI.

Build

`make image` and the scripts take:

Variable	Default	Used by
`IMAGE`	`vnap:latest`	`scripts/build/docker-build.sh`
`VANETZA_NAP_DIR`	`~/vanetza-nap`	build, `make msgcheck`
`NO_BUILD`	unset	`1`: prepare the vanetza-nap sources only
`OUTPUT_DIR`	`/vnap-certs/certify`	`scripts/certs/gen-certify.sh`

Container environment

docs/reference/container-environment.md

The patched image's entrypoint (`entrypoint.sh`, part of the patch set) starts socktap from these environment variables. `vnapctl` sets them from the scenario. They matter when starting a station by hand or writing another harness.

Variable	Meaning
`SECURITY=certs`	run socktap with `--certificate $AT_CERT --certificate-key $AT_KEY --certificate-chain $AA_CERT --trusted-certificate $ROOT_CERT`
`VANETZA_SECURITY`	`certs-v2` (default when `SECURITY=certs`) or `certs-v3`; must match the certificate format
unset `SECURITY`	socktap runs with `config.ini` (`security=none`), or with whatever `VANETZA_SECURITY` selects
`SECURITY=pseudonyms`	pool of ATs `PSEUDO_CERT_0`/`PSEUDO_KEY_0`, `PSEUDO_CERT_1`/`PSEUDO_KEY_1`, … (consecutive pairs from 0), plus `AA_CERT`/`ROOT_CERT`; starts with index 0 and changes only on control channel events; works with `certs-v2` and `certs-v3`
`PSEUDO_CONTROL_BROKER`	MQTT broker of the pseudonym control channel; without it the pseudonym never changes
`PSEUDO_CONTROL_PORT`	control broker port (1883)
`PSEUDO_CONTROL_TOPIC`	control topic prefix (default `vnap/pseudonym/<station id>`)
`PSEUDO_CONTROL_USERNAME` / `PSEUDO_CONTROL_PASSWORD`	control broker account (the password is read from the environment, not passed on the command line)
`PSEUDO_MIN_INTERVAL`	minimum milliseconds between two pseudonym changes (1000); earlier events are rejected
`PSEUDO_ID_CHANGE`	`full` (default): a pseudonym change also changes the GN address, MAC and CAM `stationId` through the ETSI ID change notification; `certificate`: only the certificate
`PSEUDO_SILENT_MIN_MS` / `PSEUDO_SILENT_MAX_MS`	random radio silence after each full ID change, in ms (default 0 = off); see pseudonym change
`PKI_REFILL_AT`	certificate refill: request a new batch from the run's PKI when this many unused pseudonyms are left (unset or 0: fixed pool that wraps around). With refill no pseudonym is used twice; uses the pseudonym control broker; see certificate refill
`PKI_BATCH_SIZE`	certificates to request per batch (8)
`PKI_TOPIC`	refill topic prefix: requests on `<prefix>/request`, batches on `<prefix>/batch` (default `vnap/pki/<station id>`)
`PKI_CATERPILLAR_KEY` / `PKI_EXPANSION_KEY`	the vehicle's butterfly secrets: with them the station derives its ATs' private keys and the PKI sends offsets instead of keys
`POSITION_CONTROL_BROKER`	MQTT broker of the position control channel: the station's position follows updates on its position topic (vehicle movement); needs the static position provider (`VANETZA_USE_HARDCODED_GPS=true`, the default), not gpsd
`POSITION_CONTROL_PORT`	position broker port (1883)
`POSITION_CONTROL_TOPIC`	position topic (default `vnap/position/<station id>`)
`POSITION_CONTROL_USERNAME` / `POSITION_CONTROL_PASSWORD`	position broker account (the password is read from the environment)
`VANETZA_LATITUDE` / `VANETZA_LONGITUDE`	position at startup; with a position channel, until the first update. `vnapctl` sets the scenario's position, the first waypoint, or the default origin in Coimbra (the image's own `config.ini` default is 40 / −8)
`VANETZA_BRIDGE_IP`	the station's V2X address; the entrypoint bridges the interface that carries it (default `eth0`). Needed when the container is on several networks, because Docker does not guarantee which one becomes `eth0`; `vnapctl` sets it

Variable	Meaning
`SUPPORT_MAC_BLOCKING=true`	bridge the link interface as `br0` (`vnapctl` sets it)

Private keys must be PKCS#8 DER (or PEM for v3); see certificates.

Stock images. For an unpatched release2-main image (`make origs`), the scenario key `entrypoint = "stock"` mounts `sim/stock-entrypoint.sh` instead. It runs the stock entrypoint's setup, then starts socktap with the certificate options the stock entrypoint cannot pass.

Control channels

docs/reference/control-channels.md

Stations with pseudonyms, movement or refill have a second network interface on the run's control network (`vnapctl0`, or `vnapctl0-iN`). On it, socktap keeps its own MQTT connection to the control broker (`pseudo-broker`), separate from the station's embedded broker that carries application messages. RSUs and the eavesdropper are not on the control network.

Channel	Topics (default prefix)	Publisher → subscriber
Pseudonym change	`vnap/pseudonym/<station id>/change`, `…/status`	event client, mobility client (mix zones), service → station; answers back
Position	`vnap/position/<station id>` (retained)	mobility client, service → station
Certificate refill	`vnap/pki/<station id>/request`, `…/batch`	station → per-run PKI; batches back

`<station id>` is the configured station ID, which stays the same across ID changes, so a station keeps its topics.

Pseudonym change

Event on `<prefix>/change` (JSON, all members optional):

```
{"event_id": "c1-7", "index": 3, "reason": "periodic"}
```

- Without `index`, the station moves to the next pseudonym (the pool wraps around unless refill is on).
- `action` (default `change`), mapping to the ETSI ID management services:
  - `{"action": "lock", "duration": 30}`: ID-LOCK (0–255 s); the answer carries a `lock_handle`.
  - `{"action": "unlock", "lock_handle": 1}`: ID-UNLOCK.
  - `{"action": "trigger"}`: IDCHANGE-TRIGGER (change to the next pseudonym).

Answer on `<prefix>/status`:

```
{"event_id": "c1-7", "result": "changed", "previous": 2, "index": 3, "pool_size": 8, "certificate":
    "<HashedId8>"}
```

Otherwise `"result": "rejected"` with an `error`, or `locked` / `unlocked` for the lock actions. Rejected events:

- an empty payload or malformed JSON (`{}` is a valid event);
- an index outside the pool, or the index already in use;
- events closer together than `PSEUDO_MIN_INTERVAL` (`pseudonyms.min_interval_ms`);
- changes while an ID-LOCK is held (`"error": "ID locked (ID-LOCK)"`);
- with refill, changes when no unused pseudonym is left;
- retained messages replayed by the broker when socktap (re)subscribes, so a stale event has no effect.

Event client (`vnap-pseudo-ctl` image, container `pseudo-client`). Its scenario settings are under `[control.client]`: `mode` = `periodic` (`interval`), `random` (`min_interval`, `max_interval`), `once` (`delay`, `index`) or `manual`; `stations`. By hand:

```
docker exec pseudo-client change 2          # station 2: next pseudonym
docker exec pseudo-client change 2 5        # station 2: pool index 5
docker exec pseudo-client ctl lock 2 20     # station 2: ID-LOCK for 20 s (the answer has the handle)
docker exec pseudo-client ctl unlock 2 1    # ID-UNLOCK handle 1
docker exec pseudo-client ctl trigger 2     # IDCHANGE-TRIGGER
docker logs -f pseudo-client                # events sent and the stations' answers
```

The service's *Control* tab and `POST /api/runs/<id>/control` send the same events and return the station's answer.

Position

Payload on `vnap/position/<station id>`:

```
{"lat": 40.208206, "lon": -8.4197756, "speed": 13.9, "heading": 90}
```

- **Fields:** `lat`/`lon` (degrees) are required. Optional: `speed` (m/s, 0–163.82), `heading` (degrees clockwise from north) and `alt` (m).
- **Rejected updates** are logged (`[MOBILITY] position update rejected: …`, the first 5).
- **Retained:** the mobility client publishes with the retain flag, so a reconnecting station gets its current position at once.

Move a station by hand:

```
docker run --rm --network vnapctl0 eclipse-mosquitto:2 mosquitto_pub -h pseudo-broker \
    -t vnap/position/2 -m '{"lat": 40.208606, "lon": -8.4197756, "speed": 10, "heading": 0}'
```

Certificate refill

- **Request** (station → PKI) on `vnap/pki/<station id>/request`: `{"request_id": "2-3", "unused": 2, "count": 8}`. The request ID is the station ID followed by the request number.
- **Batch** (PKI → station) on `vnap/pki/<station id>/batch`, answering one request:
  - the request ID and the station's i-period;
  - the certificates;
  - with station key derivation, the offsets and indices from which the station derives the private keys; otherwise the keys.
- **One request at a time:** a station has at most one request outstanding and repeats it after 10, 20, 40 … s (up to 120 s).

Details are in certificate refill.

Security

Whoever can publish on these topics controls the station:

- when it changes pseudonym, which helps an attacker correlate a change with a location, or exhaust a small pool;
- where it is;
- which certificates it receives (they are chain-checked before use).
- **Network:** only the run's own containers are on its control network.
- **Broker authentication:** `[control] auth` names environment variables holding a username and password. The service generates per-run credentials that users never see.
- **No TLS**, and one account shared by all participants of a run. Use TLS and per-client ACLs before anything outside the simulation.
- socktap logs a warning when it connects without credentials.

API

docs/reference/api.md

The service's HTTP API. The OpenAPI 3.1 description is authoritative for request and response bodies; this page gives the overview.

- **In the repository:** `openapi.json`, for reading API changes in diffs or generating clients without a running service.
- **From the running service:** `/api/openapi.json`, with an interactive view at `/api/docs`.
- **Keeping the file current:** after changing the API, run `make openapi`. `make test` fails when the file no longer matches the code.

Authentication

- **Browser sessions:** `POST /api/auth/login` with `username`, `password` and, when the account has TOTP, `totp_code`. A missing code gives 401 with `"totp_required": true`.
  - Over plain HTTP with secure cookies, login is refused with an explanation (except on localhost).
  - Login sets an HttpOnly session cookie and returns a CSRF token. State-changing requests must send it as `X-CSRF-Token`.
- **API tokens:** `Authorization: Bearer vnap_…`, created under Account or with `POST /api/tokens`. They act with the user's role, need no CSRF token and no second factor, and are revocable. TOTP settings cannot be changed with a token.
- **Roles:**
  - `viewer`: sees runs shared with it;
  - `user`: starts templates and own scenarios within the limits;
  - `admin`: everything, including the scenario catalogue, users, backups and the audit log.

Endpoints

Method and path	Who	Purpose
`POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`	all	session
`POST /api/auth/password`	all	change own password (ends all sessions)
`GET /api/auth/totp`	all	own TOTP state, recovery codes left
`POST /api/auth/totp/setup`, `/enable`, `/disable`, `/recovery-codes`	session	TOTP second factor (service deployment)
`GET/POST /api/tokens`, `DELETE /api/tokens/{id}`	all	own API tokens
`GET/POST /api/users`, `PATCH /api/users/{name}`, `POST /api/users/{name}/password`	admin	accounts: create, role, disable, unlock, reset TOTP, reset password
`GET /api/scenarios`, `GET /api/scenarios/{kind}/{name}`	all	templates, saved templates (and, for admins, the catalogue, controls as `controls/<name>`); the text of a template or catalogue scenario
`POST /api/scenarios/parse`	all	scenario text as JSON (TOML syntax only, no validation)
`POST /api/templates`	users	save scenario text as a template of one's own: `name`, `scenario_text`, `shared` (readable by every user), `genericize` (generic station and mix zone names), `replace`
`GET/DELETE /api/templates/{id}`	readers / owner	a saved template's text; delete it

Method and path	Who	Purpose
`GET /api/schema`	all	JSON Schema of the scenario format
`POST /api/scenarios/validate`	all	field-level validation against the user policy; `variant` as for runs
`GET/POST /api/runs`, `GET/DELETE /api/runs/{id}`	all	list, start (template, catalogue or scenario text, `variant`, overrides, duration, name), details with live status, delete. A variant's overrides are stored in front of the run's own
`PATCH /api/runs/{id}`	owner	change the run's `name` (at most 100 characters; empty: the scenario's name again) and `description` (at most 2000)
`POST /api/runs/{id}/stop`, `POST /api/runs/{id}/share`	owner	stop; share read-only with another user
`GET /api/runs/{id}/layout`	readers	description, stations, routes, crossings, mix zones (for the map)
`GET /api/runs/{id}/events`, `GET /api/runs/{id}/events/stream`	readers	events from the logs (optionally live MQTT for up to 30 s); server-sent events: the last `since_s` seconds (default 5, at most 3600), an event `ready`, then new events
`GET /api/runs/{id}/positions`	readers	current position of every moving station
`GET /api/runs/{id}/eavesdropper`	readers	the eavesdropper's report and the score against ground truth
`POST /api/runs/{id}/control`	owner	pseudonym change, trigger, lock, unlock; returns the station's answer
`POST /api/runs/{id}/stations/{station}/position`	owner	move a station
`POST /api/runs/{id}/check`, `GET /api/runs/{id}/checks`	owner / readers	run a check (verdict, metrics); earlier checks
(run ids)		`<user>-<scenario name>-<n>`, see runs
`GET /api/runs/{id}/results`, `GET /api/runs/{id}/results/{name}`	readers	files collected at stop, among them the summary `report.docx`
`GET/POST /api/backups`	admin	list archives; back up now
`GET /api/admin/certs`	admin	the committed test certificate sets: each certificate's validity, missing files, `generated.json`, the lifetimes with their defaults and basis, the last regeneration job
`POST /api/admin/certs/{v2|v3}/regenerate`	admin	`{"lifetimes": {"at_hours": 8760, ...}}` (keys left out take their defaults); 202 with the job, which runs in the background; 400 for a lifetime out of range or longer than its signer's, 409 while another runs (certificates)
`GET /api/audit`	admin	audit log (`limit`, `username`)
`GET /api/ui-config`	public	map tiles and origin for the web UI
`GET /api/healthz`	public	liveness

"Readers" are the owner, the users a run is shared with, and admins. Others get 404 for runs they cannot see.

Errors

- **Validation errors:** 400 with `errors`, a list of `{path, message, text}`.
- **Refused by limits:** 429.
- **Control channel unreachable:** 502.
- **Other errors:** a JSON body with `detail` (or `error`).

Example

```
T=vnap_…                                              # API token
curl -s -H "Authorization: Bearer $T" https://vnap.example.org/api/scenarios
RUN=$(curl -s -H "Authorization: Bearer $T" -H 'Content-Type: application/json' \
  -d '{"template": "refill", "duration_minutes": 10}' https://vnap.example.org/api/runs | jq -r .id)
curl -s -H "Authorization: Bearer $T" -H 'Content-Type: application/json' -d '{"duration_s": 15}' \
  https://vnap.example.org/api/runs/$RUN/check | jq .verdict
curl -s -X POST -H "Authorization: Bearer $T" https://vnap.example.org/api/runs/$RUN/stop
```

Scenarios

sim/scenarios/README.md

Five foundational scenarios, one per phenomenon: `trust`, `linkability`, `crossing`, `convoy` and `refill`. Each has variants (`[variants.<name>]`, started as `../vnapctl up crossing:min3`). `controls/` holds pass/fail controls; `templates/` links the scenarios the service offers its users. `../vnapctl scenarios` lists them all.

- **The format:** docs/reference/scenario-format.md (also `../vnapctl schema`).
- **Which scenario does what:** docs/operations/vnapctl.md.

Patch set

Patch documentation

docs/patches/README.md

The patch set changes Vanetza-NAP's `socktap` tool, parts of the Vanetza library and the image build. It applies to upstream vanetza-nap `release2-main` at the commit in `patches/vanetza-nap/BASE` (`241438fd`).

- `patches/vanetza-nap/modified/`**:** the patched files, each named after its path with `/` written as `__`.
- `patches/vanetza-nap/upstream/`**:** the upstream originals of the files that existed before.
- `patches/vanetza-nap/diffs/`**:** generated unified diffs (`make diffs`).
- **Building:** `make image` lays the patched files over the base and builds `vnap:latest` (build).
- **Commit series:** the same changes exist as commits on the local vanetza-nap branch `jodyhuntatx`, which is not published.

Patches

Group	Patch	Why	Upstream status
Build and container	Debian snapshot mirror	bullseye reached end of life; its pool files return 404, so the image no longer built	candidate
	`certify` and certificate mode in the entrypoint	ship `certify`; start socktap with certificate, chain and trusted root from the environment	candidate
	Bridge interface selection	bridge the interface carrying `VANETZA_BRIDGE_IP`, not whatever became `eth0`	candidate
Certificate verification	v3 full-chain verification	upstream v3 accepted any AT regardless of issuer	candidate
	Startup certificate checks	log chain results for the configured AA and own ATs	candidate
	Optional Assurance_Level (v2)	accept older v2 certificates without the attribute	simulation convenience
	v3 DER keys	load PKCS#8 DER keys; readable errors	candidate
Thread safety	Clock sync, PRNG, sign header policy, v2/v3 certificate caches, runtime clock and ASN.1 descriptors, routers, PubSub/MQTT/DDS, RSSI reader	data races found with ThreadSanitizer; several crashed socktap under load	candidate
Pseudonym and ID change	Pseudonym pool	a station holds a pool of ATs (e.g. a butterfly batch)	simulation-specific
	Event-driven pseudonym change	changes on events from a separate MQTT control channel, not on a timer	simulation-specific
	ID change notification service	ETSI TS 102 723-8/-9: GN address, MAC and `stationId` change with the certificate; ID-LOCK	standards-based; candidate for discussion
	Silent period	random radio silence after each full ID change (TR 103 415)	simulation-specific
Certificate refill	Growing pool and PKI channel	new ATs from the run's PKI at a threshold; never reused	simulation-specific
	Station key derivation	butterfly key expansion on the station (IEEE 1609.2.1)	simulation-specific
Mobility and CAM	Position control channel	position from an MQTT topic at runtime	simulation-specific

Group	Patch	Why	Upstream status
	CAM kinematics	heading, speed and heading confidence, acceleration, yaw rate in the CAM's units	candidate
	CAM timer rephase	the CAM timer restarts at a random phase on every full ID change	simulation-specific

"Candidate" means the change fixes upstream behaviour and could be proposed upstream as is. "Simulation-specific" means it exists for this study (control channels, PKI protocol).

Files

Every patched file, and the groups that change it. `make diffs-check` fails if a patched file is missing here.

File	New	Groups
`Dockerfile`		build and container
`entrypoint.sh`		build and container; pseudonym and ID change; certificate refill; mobility and CAM
`tools/socktap/CMakeLists.txt`		pseudonym and ID change; certificate refill; mobility and CAM
`tools/socktap/main.cpp`		thread safety; pseudonym and ID change; certificate refill; mobility and CAM
`tools/socktap/security.cpp`		certificate verification; pseudonym and ID change; certificate refill
`tools/socktap/security.hpp`		pseudonym and ID change; certificate refill
`tools/socktap/applications/cam_application.cpp`		thread safety; pseudonym and ID change; mobility and CAM
`tools/socktap/applications/cam_application.hpp`		mobility and CAM
`tools/socktap/time_trigger.cpp`		thread safety; pseudonym and ID change
`tools/socktap/time_trigger.hpp`		thread safety; pseudonym and ID change
`tools/socktap/router_context.cpp`		thread safety; pseudonym and ID change
`tools/socktap/router_context.hpp`		thread safety; pseudonym and ID change
`tools/socktap/dcc_passthrough.cpp`		thread safety; pseudonym and ID change
`tools/socktap/dcc_passthrough.hpp`		thread safety; pseudonym and ID change
`tools/socktap/raw_socket_link.cpp`		thread safety
`tools/socktap/raw_socket_link.hpp`		thread safety
`tools/socktap/pubsub.cpp`		thread safety
`tools/socktap/dds.cpp`		thread safety
`tools/socktap/mqtt.cpp`		thread safety
`tools/socktap/mqtt.hpp`		thread safety
`tools/socktap/rssi_reader.cpp`		thread safety
`tools/socktap/positioning.cpp`		mobility and CAM
`tools/socktap/mobility.cpp`	new	mobility and CAM
`tools/socktap/mobility.hpp`	new	mobility and CAM
`tools/socktap/pseudonym_channel.cpp`	new	pseudonym and ID change
`tools/socktap/pseudonym_channel.hpp`	new	pseudonym and ID change
`tools/socktap/id_change.cpp`	new	pseudonym and ID change
`tools/socktap/id_change.hpp`	new	pseudonym and ID change
`tools/socktap/pki_channel.cpp`	new	certificate refill
`tools/socktap/pki_channel.hpp`	new	certificate refill

File	New	Groups
tools/socktap/bke.cpp	new	certificate refill
tools/socktap/bke.hpp	new	certificate refill
vanetza/asn1/asn1c_wrapper.hpp		thread safety
vanetza/asn1/cam.hpp		thread safety
vanetza/common/manual_runtime.cpp		thread safety
vanetza/common/manual_runtime.hpp		thread safety
vanetza/security/CMakeLists.txt		certificate verification; pseudonym and ID change; certificate refill
vanetza/security/backend_cryptopp.cpp		thread safety
vanetza/security/backend_cryptopp.hpp		thread safety
vanetza/security/straight_verify_service.cpp		certificate verification
vanetza/security/straight_verify_service.hpp		certificate verification
vanetza/security/id_change_service.cpp	new	pseudonym and ID change
vanetza/security/id_change_service.hpp	new	pseudonym and ID change
vanetza/security/pseudonym_control.hpp	new	pseudonym and ID change; certificate refill
vanetza/security/pseudonym_pool.hpp	new	certificate refill
vanetza/security/v2/certificate_cache.cpp		thread safety
vanetza/security/v2/certificate_cache.hpp		thread safety
vanetza/security/v2/default_certificate_validator.cpp		certificate verification
vanetza/security/v2/sign_header_policy.cpp		thread safety
vanetza/security/v2/sign_header_policy.hpp		thread safety
vanetza/security/v2/pseudonym_certificate_provider.cpp	new	pseudonym and ID change; certificate refill
vanetza/security/v2/pseudonym_certificate_provider.hpp	new	pseudonym and ID change; certificate refill
vanetza/security/v3/certificate_cache.cpp		thread safety
vanetza/security/v3/certificate_cache.hpp		thread safety
vanetza/security/v3/certificate_chain.cpp	new	certificate verification
vanetza/security/v3/certificate_chain.hpp	new	certificate verification
vanetza/security/v3/persistence.cpp		certificate verification
vanetza/security/v3/sign_header_policy.cpp		thread safety
vanetza/security/v3/sign_header_policy.hpp		thread safety
vanetza/security/v3/pseudonym_certificate_provider.cpp	new	pseudonym and ID change; certificate refill
vanetza/security/v3/pseudonym_certificate_provider.hpp	new	pseudonym and ID change; certificate refill

Working on the patch set

1. Edit in a vanetza-nap tree (`~/vanetza-nap`), build with `make image`, test with scenarios.
2. Copy the changed files into `patches/vanetza-nap/modified/` (path with `/` as `__`); for a file that existed upstream, its original goes into `upstream/`.
3. `make diffs`, then add the file to the table above and describe the change in its group document.
4. `make test`, then a live run of the scenarios the change affects.

Patches: build and container

docs/patches/build-and-container.md

Debian snapshot mirror

- **Problem:** the image's build stages use Debian bullseye, which reached end of life on 2026-08-31. `deb.debian.org` pool files then returned 404, so the image no longer built.
- **Change:** in both build stages, `Dockerfile` switches apt to the Debian snapshot sources that the base image ships commented out (`snapshot.debian.org`, without the validity check).
- **Upstream:** candidate (any build of release2-main hits it).

`certify` and certificate mode

- **Problem:** the stock image does not ship Vanetza's `certify` tool, and its entrypoint cannot pass certificate options to socktap.
- **Change:**
  - `Dockerfile` builds `certify` (`-DBUILD_CERTIFY=ON`) and copies it into the image;
  - `entrypoint.sh` starts socktap with certificate, AA chain and trusted root from environment variables (`SECURITY=certs`, `AT_CERT`, `AT_KEY`, `AA_CERT`, `ROOT_CERT`, `VANETZA_SECURITY`).

  The later patch groups add their own variables (pseudonym pool, control channels, refill); all are listed in container environment.
- **Upstream:** candidate.

Bridge interface selection

- **Problem:** a station on two networks (message and control) gets two interfaces, and Docker does not guarantee which one becomes `eth0`. The stock entrypoint bridged `eth0`, so V2X traffic could end up on the control network.
- **Change:** `entrypoint.sh` bridges the interface that carries `VANETZA_BRIDGE_IP` (the station's V2X address), falling back to `eth0`. `vnapctl` sets it.
- **Upstream:** candidate.

Files

`Dockerfile`, `entrypoint.sh`.

Testing

- `make image` builds.
- Every scenario starts its stations through this entrypoint.
- `entrypoint = "stock"` scenarios use an unpatched image with `sim/stock-entrypoint.sh`, for comparison.

Patches: certificate verification

`docs/patches/certificate-verification.md`

How the stations use these checks is described in security and chain verification.

v3 full-chain verification

- **Problem:** upstream v3 verification accepted a message signed by any AT, regardless of its issuer, and `--trusted-certificate` had no effect for v3.
- **Change:**
  - a new `vanetza/security/v3/certificate_chain.{hpp,cpp}` verifies AT → AA → trusted root, with the IEEE 1609.2 signing input;
  - `straight_verify_service.{hpp,cpp}` uses it for received messages;
  - `tools/socktap/security.cpp` loads the AA chain and the trusted root for v3.
- **Effect:** messages from ATs of another hierarchy fail with a certificate error. `trust:badroot` (OBU trusts the TLM certificate) and `controls/naive-v3` (self-made certificates) are the negative controls.
- **Upstream:** candidate.

Startup certificate checks

- **Change:** `tools/socktap/security.cpp` logs the result of checking the configured AA and the station's own AT(s) at startup: `[V3-CHAIN]` / `[V2-CHAIN]` lines. Each line is a single write, because other threads log at the same time. The refill patches reuse it for every AT that arrives.
- **Upstream:** candidate.

Optional Assurance_Level (v2)

- **Problem:** TS 103 097 V1.2.1 requires the `Assurance_Level` attribute, and upstream rejects v2 certificates without it. Older v2 certificates lack it.
- **Change:** `vanetza/security/v2/default_certificate_validator.cpp` accepts its absence. Current C-ITS-PKI and `certify` output include it, so this matters only for old certificate sets.
- **Upstream:** simulation convenience; not proposed.

v3 DER keys

- **Problem:** v3 could not load PKCS#8 DER private keys, and failures ended in `terminate … char const*` without a message.
- **Change:** `vanetza/security/v3/persistence.cpp` loads PKCS#8 DER keys and reports readable errors.
- **Upstream:** candidate.

Files

- `vanetza/security/v3/certificate_chain.{hpp,cpp}` (new);
- `vanetza/security/straight_verify_service.{hpp,cpp}`;
- `vanetza/security/CMakeLists.txt`;
- `vanetza/security/v2/default_certificate_validator.cpp`;
- `vanetza/security/v3/persistence.cpp`;
- `tools/socktap/security.cpp`.

Testing

Every secured scenario checks success rates and chain results (`vnapctl check` default expectations). The negative controls check that wrong chains are rejected. `vnap:msgcheck` runs the same verification on message files offline (troubleshooting).

Patches: thread safety

docs/patches/thread-safety.md

socktap receives and verifies packets on several threads, signs and sends on others, and handles MQTT, DDS, timers and RSSI polling on still others. Running a sanitized build (ThreadSanitizer) under load, mostly with `controls/stress-naive-v3` (4 stations at 50 Hz), found the races below. Several crashed socktap. The number of reports went from 69 to 17 in socktap and Vanetza code, then to none. The remaining reports are inside the Zenoh library's uninstrumented Rust threads and are most likely false positives (troubleshooting).

Patch	Files	Race and fix
Clock sync	`tools/socktap/time_trigger.{hpp,cpp}`	the runtime clock lagged wall time, so CAMs were rejected as `Invalid_Timestamp`; also a concurrent `schedule()` assertion crash. A 10 ms sync pulse keeps the runtime clock within 10 ms of wall time, and scheduling is serialized by a recursive mutex (the pulse only try-locks, keeping the `io_context` thread unblocked)
PRNG mutex	`vanetza/security/backend_cryptopp.{hpp,cpp}`	the shared CryptoPP random pool was used from several threads (v3 verification, key-check asserts); now under a mutex
Sign header policy	`vanetza/security/v{2,3}/sign_header_policy.{hpp,cpp}`	verification on several threads and signing on another accessed the policy's P2P request trackers unsynchronized, which aborted socktap (`PeerRequestTracker` assertion) under load; now locked. Reproduce with `controls/stress-naive-v3`
v3 certificate cache	`vanetza/security/v3/certificate_cache.{hpp,cpp}`	reception threads stored certificates while others looked them up (data races and a SEGV in lookup); locked, and the short-digest index keeps pointers instead of rehash-invalidated iterators
v2 certificate cache	`vanetza/security/v2/certificate_cache.{hpp,cpp}`	concurrent inserts and lookups, and lookups modify the cache (expiry heap): data races and a corrupted heap (boost assertion abort); locked
Runtime clock and ASN.1 descriptors	`vanetza/common/manual_runtime.{hpp,cpp}`, `vanetza/asn1/asn1c_wrapper.hpp`, `vanetza/asn1/cam.hpp`, `tools/socktap/applications/cam_application.cpp`	reception threads read the runtime clocks while the main thread advances them (now atomic); `asn1c_wrapper_common::swap` wrote to the global type descriptors; `CompactR2DecodeGuard` swaps a global R2 CAM member descriptor, now under a process-wide recursive mutex that all CAM encode/decode/JSON paths take. Reports 69 → 38
Routers	`tools/socktap/{router_context,time_trigger,dcc_passthrough,raw_socket_link}.{hpp,cpp}`, `pubsub.cpp`, `main.cpp`	each reception router was used by its reception thread, by timers and position updates on the main thread, and by the PubSub transmission thread. Now each router is guarded by its trigger's lock (main-thread timers only try-lock), worker threads wait for `RouterContext::start()`, the thread-to-trigger map is locked (new triggers created outside that lock, avoiding a lock-order inversion) and the link-layer callback is published safely. Reports 38 → 19

Patch	Files	Race and fix
PubSub, MQTT and DDS	`tools/socktap/{pubsub,dds,mqtt}.cpp`, `mqtt.hpp`	MQTT subscriptions were added while the mosquitto loop thread iterated them; callback threads inserted into the topic priority map; several threads used one UDP socket; DDS `operator[]` lookups inserted (and dereferenced) null publishers. Maps locked, lookups use `find()`, UDP sends serialized; also reads MQTT payloads by length instead of as NUL-terminated strings. Reports 19 → 17
RSSI reader	`tools/socktap/rssi_reader.cpp`	the RSSI thread (nl80211 polling) inserts into and expires the RSSI/MCS maps and writes the channel survey while the receive thread reads them for every packet; on a real radio this could crash socktap. One mutex, never held across netlink I/O. Reports 2 → 0

Upstream: all candidates. They fix crashes and undefined behaviour that are independent of this study.

Testing

- `controls/stress-naive-v3` must run its full duration without a crash line (`check`'s `errors.total`).
- **ThreadSanitizer:** to repeat the analysis, build socktap with `-fsanitize=thread` in a vanetza-nap tree, run a station with it in a scenario, and read the reports from its log.

Patches: pseudonym and ID change

`docs/patches/pseudonym-and-id-change.md`

The behaviour is described in pseudonym change, and the channel in control channels.

Pseudonym pool

- **Change:** new `vanetza/security/v{2,3}/pseudonym_certificate_provider.{hpp,cpp}` hold a pre-provisioned pool of ATs (e.g. a butterfly batch) for v2 and v3. After each change, the full new certificate is sent in the next message, so receivers learn it at once.
- **Options:** `--pseudonym-certificate` and `--pseudonym-certificate-key`, repeatable and paired in order; `SECURITY=pseudonyms` with `PSEUDO_CERT_<i>` / `PSEUDO_KEY_<i>` in the entrypoint.
- **Logs:** `[PSEUDONYM]` lines (start, changes, rejections).
- **Files:** the providers, `vanetza/security/CMakeLists.txt`, `tools/socktap/security.{hpp,cpp}`, `entrypoint.sh`.

Event-driven pseudonym change

- **Change:** the pseudonym changes only on events from a separate MQTT control channel, not on a timer.
  - `vanetza/security/pseudonym_control.hpp` (new) is the interface the providers implement.
  - `tools/socktap/pseudonym_channel.{hpp,cpp}` (new) is socktap's own MQTT client.
  - `time_trigger.{hpp,cpp}` gains `post()`, so the change runs on the `io_context` under the lock that also guards signing.
- **Options:** `--pseudonym-control-broker`, `-port`, `-topic`, `-username`, `-password`, and `--pseudonym-min-interval` (`PSEUDO_CONTROL_*`, `PSEUDO_MIN_INTERVAL`).
- **Files:** `pseudonym_control.hpp`, `pseudonym_channel.{hpp,cpp}`, `tools/socktap/main.cpp`, `tools/socktap/CMakeLists.txt`, `time_trigger.{hpp,cpp}`, `entrypoint.sh`.

ID change notification service

- **Change:** ETSI TS 102 723-8/-9 ID change notification. The parts:
  - `vanetza/security/id_change_service.{hpp,cpp}` (new): subscribe, two-phase commit (PREPARE/COMMIT/ABORT/DEREG), trigger, ID-LOCK/UNLOCK;
  - `tools/socktap/id_change.{hpp,cpp}` (new): socktap's subscribers, for the network layer (routers: GN address and MAC) and the facilities layer (CAM `stationId`);
  - the providers and `pseudonym_control.hpp`: they run the change through the service;
  - `router_context.{hpp,cpp}`: locking routers during PREPARE and setting the new address on COMMIT;
  - `pseudonym_channel.cpp`: the `lock`, `unlock` and `trigger` actions.
- **Options:** `--pseudonym-id-change full|certificate` (`PSEUDO_ID_CHANGE`, default `full`).
- **Logs:** `[IDCHANGE]` lines (network, facilities, lock, unlock).
- **Files:** `id_change_service.{hpp,cpp}`, `pseudonym_control.hpp`, the providers, `vanetza/security/CMakeLists.txt`, `id_change.{hpp,cpp}`, `router_context.{hpp,cpp}`, `main.cpp`, `pseudonym_channel.cpp`, `applications/cam_application.cpp`, `tools/socktap/CMakeLists.txt`, `entrypoint.sh`.
- **Upstream:** standards-based; candidate for discussion upstream.

Silent period

- **Change:** after each full ID change, `DccPassthrough` (`tools/socktap/dcc_passthrough.{hpp,cpp}`) drops all outgoing frames for a random time (ETSI TR 103 415 clause 4.1.4), started by the network layer on COMMIT.
- **Options:** `--pseudonym-silent-min` / `--pseudonym-silent-max` (`PSEUDO_SILENT_MIN_MS` / `PSEUDO_SILENT_MAX_MS`).
- **Logs:** `[IDCHANGE] silent period <n> ms`, then `silent period over, <n> frame(s) suppressed`.

Testing

- **Scenarios:**
  - `linkability` and `linkability:manual`: changes, rejections, answers;
  - `linkability`, `linkability:silent` and `linkability:certificate-only`: an eavesdropper against full ID changes, with and without silence, and against certificate-only changes;
  - `crossing` and `convoy` with their variants (results).
- **Checks:** `<station>.pseudonym.changed`, `.rejected`, `control.changed`, and `vnapctl events --kind idchange`.

Patches: certificate refill

docs/patches/certificate-refill.md

The behaviour is described in certificate refill, and the protocol in control channels.

Growing pool and PKI channel

- **Change:** the pseudonym pool grows at runtime.
  - `vanetza/security/pseudonym_pool.hpp` (new) keeps the pool, tracks unused entries and never wraps around when refill is on.
  - `tools/socktap/pki_channel.{hpp,cpp}` (new) requests batches at a threshold of unused ATs, retries with back-off, and installs the answer while running.
  - Each new AT is chain-checked before use (`[V3-CHAIN] batch authorization ticket …`).
  - With no unused AT left, changes are refused until a batch arrives.
- **Options:** `--pki-refill-at`, `--pki-batch-size`, `--pki-topic`, `--pki-retry` (`PKI_REFILL_AT`, `PKI_BATCH_SIZE`, `PKI_TOPIC`).
- **Logs:** `[PKI] batch requested …`, `[PKI] batch installed … refresh <ms> ms …`, retries.
- **Files:** `pseudonym_pool.hpp`, `pseudonym_control.hpp`, `vanetza/security/v{2,3}/pseudonym_certificate_provider.{hpp,cpp}`, `pki_channel.{hpp,cpp}`, `tools/socktap/security.{hpp,cpp}`, `main.cpp`, `tools/socktap/CMakeLists.txt`, `vanetza/security/CMakeLists.txt`, `entrypoint.sh`.

Station key derivation

- **Change:** `tools/socktap/bke.{hpp,cpp}` (new) implements the IEEE 1609.2.1 butterfly expansion function f_k(i, j): three AES-128 blocks, each `AES_k(x+t) XOR (x+t)`, read as a big-endian integer mod n. The input x is the 32-bit prefix, i, j and 32 zero bits.
  - The station computes each AT's private key as a + f_k(i, j) + r mod n from its caterpillar key a, its expansion key k and the offset r the PKI sends.
  - `security.cpp` checks that every derived key matches its certificate's public key (v2 and v3), and rejects the AT otherwise.
- **Options:** `--pki-caterpillar-key`, `--pki-expansion-key` (`PKI_CATERPILLAR_KEY`, `PKI_EXPANSION_KEY`).
- **Logs:** the batch installed line adds `key derivation <ms> ms`.
- **Files:** `bke.{hpp,cpp}`, `pki_channel.{hpp,cpp}`, `security.cpp`, `entrypoint.sh`.

Testing

- **Scenarios:** `refill` with its variants, and the run's PKI in `linkability`, `crossing` and `convoy`, with the default expectations `<station>.pki.starved==0` and `pki.running==1`.
- **Measurements:** the earlier test runs are listed in results; `results/pki-refill-test-2026-10-07.docx` relates refresh time to batch size.
- **C-ITS-PKI side:** the PKI's half of the expansion is checked by C-ITS-PKI's contract test (architecture).

Patches: mobility and CAM

`docs/patches/mobility-and-cam.md`

The behaviour is described in vehicle movement and pseudonym change.

Position control channel

- **Change:**
  - `tools/socktap/mobility.{hpp,cpp}` (new) subscribes to the station's position topic on the control broker;
  - `positioning.cpp` replaces the static position provider with a thread-safe controllable one, whose fix feeds the CAM and the GeoNetworking position vector.
- **Options:** `--position-control-broker`, `-port`, `-topic`, `-username`, `-password` (`POSITION_CONTROL_*`).
- **Logs:** `[MOBILITY]` lines, including rejected updates (the first 5).
- **Files:** `mobility.{hpp,cpp}`, `positioning.cpp`, `main.cpp`, `tools/socktap/CMakeLists.txt`, `entrypoint.sh`.

CAM kinematics

- **Problem:** upstream filled the CAM's kinematic fields from a static position only, and several were wrong in units once the vehicle moved:
  - **Heading:** degrees went into a field in 0.1°.
  - **Speed and heading confidence:** m/s and degrees went into fields in 0.01 m/s and 0.1°; a confidence of 0.1 m/s became the invalid value 0, and the CAM could not be encoded.
  - **Longitudinal acceleration and yaw rate:** wrong scale; the yaw rate also had the wrong sign.
- **Change:** `tools/socktap/applications/cam_application.cpp` converts each field to the CAM's units and sign conventions.
- **Upstream:** candidate.

CAM timer rephase

- **Problem:** the CAM timer kept running through an ID change, so each car's position within the 1 s CAM cycle carried over to its new identity and linked the two.
- **Change:** on COMMIT of a full ID change, `cam_application.{hpp,cpp}` skips the next CAM and restarts the timer after a random delay of up to one interval (`[IDCHANGE] CAM timer: new phase, next CAM in <n> ms`). `main.cpp` wires the CAM application to the ID change service.
- **Effect:** a timing-only linker went from 16 of 16 changes matched to 1 of 13 in the random-turn mix zone.

Testing

- **Scenarios:** `linkability` (positions in the received CAMs), `convoy` and `crossing`.
- **Results:** the mix zone reruns on the rephased image are in results.

Results

Test summaries

`results/README.md`

Results of the privacy scenarios: what the passive eavesdropper (`sim/images/eavesdropper/`, see eavesdropper and scoring) could link across pseudonym changes, and measurements of certificate refill. Scenario files are in `sim/scenarios/`. Each result names the station image it was measured with (tags: CHANGELOG).

CAM timing leak. The eavesdropper results below, except the vnap:r2-p17 reruns and the refill measurements (vnap:r2-p18 and later), were measured with images up to vnap:r2-p16, where the CAM timer kept running through each ID change. A car's position within the 1 s CAM cycle therefore carried over to its new identity. Matching on that timing alone linked 16 of 16 changes in the random-turn mix zone, so these results hold only against an eavesdropper that ignores timing. vnap:r2-p17 restarts the CAM timer at a random phase on every ID change (the same matching then linked 1 of 13). See pseudonym change.

Scenario names. The results name the scenario files as they were when measured. Since 2026-10-18 these are five foundational scenarios with variants, e.g. `c-its-pki-mixzone` is now `crossing:blocks` and `c-its-pki-convoy` is `convoy:periodic`. The table in vnapctl maps every earlier name and says where the setup differs (several now take their certificates from the run's PKI); the earlier files are in the git history.

Documents

Document	Scenario	Result
convoy-test-2026-10-04.docx	`c-its-pki-convoy`: three OBUs driving together, synchronized full ID changes, 3–13 s silent periods, 15 s link window	of 16 links, 11 joined two different cars; recommends further scenarios
mixzone-test-2026-10-04.docx	`c-its-pki-mixzone`: four OBUs changing identity in an intersection mix zone, 6–12 s silent periods, 15 s link window	of 17 links, 15 joined two different cars, but all follow one rotation an attacker could undo; recommends random turns next
mixzone-random-test-2026-10-04.docx	`c-its-pki-mixzone-random`: the same crossing with random turns and random return roads	of 18 links, 17 joined two different cars, spread over 10 pairings; the best fixed turn rule recovers 10 of 24 crossings; recommends a turn-aware attacker next
mixzone-test-2026-10-04-r2-p17.docx	`c-its-pki-mixzone` rerun on vnap:r2-p17 (CAM timer rephase), eavesdropper linking by timing and position	18 links, none correct; timing at chance level, but 19 of 19 position links (replay) form the same one-road rotation, so a pattern-aware attacker can still undo them
mixzone-random-test-2026-10-04-r2-p17.docx	`c-its-pki-mixzone-random` rerun on vnap:r2-p17	12 links, none correct; timing at chance level and position mistakes without a pattern
pki-refill-test-2026-10-07.docx	`c-its-pki-refill-sweep` (batch sizes 4–32, four OBUs refilling together) and `c-its-pki-mixzone-pki` (random-turn mix zone with the run's own PKI)	refresh time ≈ PKI issue time (≈50 ms per ticket) × batch size × stations asking at once; no starvation, reuse or cross-run sharing; mix-zone links: 4 of 25 correct, timing at chance

Each document has the setup, every link scored against ground truth, limitations and recommendations for subsequent scenarios.

Earlier results

Shorter runs without a separate document (moved here from the earlier single README).

- **Full ID change (default):** in c-its-pki-tracking every OBU pseudonym change also changes its MAC, GN address and `stationId`, so the eavesdropper sees a new vehicle each time. It still

links all of them by position continuity: the simulated OBU never moves, and a vehicle that keeps sending its exact position is easy to follow. Unlinkability needs more than synchronized identifiers, e.g. silent periods or changes in mix zones.

- **Full ID change with a silent period** (`c-its-pki-tracking-silent`, 3–13 s):
  - **Default eavesdropper:** in a 2.5-minute run with 10 changes it linked none. It saw the one OBU as 8 separate vehicles, because each silence exceeded its 3 s link window.
  - **Patient eavesdropper:** one with `--link-window 15`, attached to the same run, linked every change again (gaps of 5–12 s). With a single stationary vehicle and nobody else around, waiting out the silence is enough.
  - **What this means:** a silent period protects only where other vehicles could be the one that reappears, i.e. in dense traffic or mix zones (TR 103 415 clauses 4.1.4–4.1.6).
  - **The cost:** the vehicle is invisible while silent; the RSU received 7 instead of 10 OBU CAMs in a 10 s window.
- **Moving vehicles** (`c-its-pki-traffic`: two OBUs on crossing roads at 40 and 50 km/h, full ID change and 3–13 s silent periods):
  - **Default eavesdropper:** linked none of the changes; every new identity looked like a new vehicle (8 vehicles plus the RSU in the first run).
  - **Patient eavesdropper (`--link-window 15`):** linked every change of both vehicles (3 of 3 each) without mixing them up. It predicts where a silent vehicle reappears from its last speed and the length of the gap, and two vehicles on different roads are easy to tell apart. Movement alone is no protection; it would take vehicles that could plausibly have swapped places during the silence (a mix zone).
- **Certificate-only changes:** with `pseudonyms.id_change = "certificate"` it links every change through the unchanged MAC, GN address and `stationId`.
- **Certificate refill** (`vnap:r2-p18-test`, two OBUs changing every 5 s, refill at 2, batches of 8, the PKI returning the private keys):
  - **Refill:** each OBU received batches as it ran low. Every new AT passed the chain check, and the RSU verified the CAMs signed with them.
  - **No reuse:** each OBU used 17 distinct ATs, all from its own entries in `issued.jsonl`.
  - **Refresh time:** 454–971 ms, of which the PKI issued a batch of 8 in 376–533 ms. Both OBUs asked at the same moment, so one waited up to 537 ms behind the other at the PKI.
  - **PKI outage:** with the PKI stopped, requests were retried after 10 s and 20 s, and four changes were refused while the pool was empty. The first retry after the PKI came back was answered (refresh 31 170 ms, counted from the first unanswered request).
  - **Fixed pools:** without `PKI_REFILL_AT` the regression scenarios pass; the pool's unit test confirms that fixed pools still wrap around.
  - **ThreadSanitizer:** the pool's own test passes clean, and a sanitized OBU running two refills reported nothing in the new code (8 reports, all inside the Zenoh library).

Appendix

Changelog

CHANGELOG.md

Features by date, station image tags, the mapping to the product requirements (PRD) phases, and the history of this repository. The documentation in docs/ describes only the current behaviour; how it got there is recorded here.

Station image tags

vnap:latest is the newest build. Tags mark images used for measurements, so results in results/ can name the image they were measured with.

Tag	Built	Adds
vnap:r2-p19 (= latest)	2026-10-07	butterfly key derivation on the station (PKI returns offsets, not keys)
vnap:r2-p18	2026-10-07	certificate refill: pseudonym pool growing at runtime from the run's PKI
vnap:r2-p17	2026-10-04	CAM timer restarts at a random phase on every full ID change
vnap:r2-p16	2026-10-04	position control channel and CAM kinematics fixes (vehicle movement)
earlier r2-p*	2026-09-28 – 10-03	release2 patch set, pseudonym pool and events, thread-safety patches, ID change notification, silent periods

Results measured with images up to r2-p16 carry the CAM timing caveat described in results/README.md: the CAM timer then kept its phase across ID changes.

2026-10-18: regenerating the test certificates

vnapctl certs shows the validity of every certificate in the committed sets (certs/c-its-pki, v3; certs/certify, v2), and vnapctl certs regenerate v2|v3 --lifetime KEY=N makes a new set: v3 with C-ITS-PKI in the vnap-pki image (sim/images/pki/test_certs.py), v2 with Vanetza's certify in the station image. Administrators do the same in the web UI (*Administration* □ *Test certificates*) or with GET /api/admin/certs and POST /api/admin/certs/{set}/regenerate. Each lifetime (root, TLM, EA, AA, AT for v3; root, AA, AT for v2) defaults to the generating tool's value, because the ETSI documents fix none, and can be changed. A certificate may not outlive its signer. The new set is checked before it replaces the old one, and generated.json records the lifetimes. The v2 tickets are now ticket_vnap_rsu and ticket_vnap_obu, without a date, so a regenerated set keeps the names the scenarios use. See certificates.

Why: the committed v3 ATs were issued on 2026-10-04 with C-ITS-PKI's default of 168 hours and expired on 2026-10-11; since then every scenario using them failed its check, with nothing in the logs.

2026-10-18: five foundational scenarios with variants

The 19 scenario files and 2 templates in sim/scenarios/ are now five foundational scenarios, one per phenomenon: trust (certificate-chain verification), linkability (what a change leaks when a vehicle is alone), crossing (privacy from other vehicles in a mix zone), convoy (vehicles that change together but never separate) and refill (certificate supply from the run's PKI). The settings studied so far are their **variants**, named sets of overrides started as vnapctl up crossing:min3, or with the variant list in the web UI and variant in the API. controls/ keeps the pass/fail controls none, naive-v3 and stress-naive-v3; c-its-pki-badroot is now trust:badroot. templates/ links the four scenarios users may start. Every earlier name maps to a scenario and variant in vnapctl.

- `--set` addresses entries of arrays of tables by name or index (`control.mix_zones.crossing.min_vehicles=3`) and replaces whole arrays (`control.mix_zones=[]`).
- `[defaults] pseudonyms` holds keys for every station with a pseudonym pool, so one override changes them all.
- `linkability`, `convoy` and the `crossing:blocks` variants now take certificates from the run's PKI instead of the committed pools, and `linkability`'s OBU moves; results measured with the earlier files are therefore not exactly reproduced by their new names.
- `make test` loads every scenario with each of its variants.

2026-10-17: documentation as one PDF

`docs/vnap-secure-documentation.pdf` collects all Markdown documentation, with an introduction on the design intention of VNAP-Secure (`scripts/docs/pdf-intro.md`); the README links it at the top. `make docs-pdf` rebuilds it (`scripts/docs/build-pdf.py`, reportlab and markdown-it in `scripts/docs/.venv`), and `make test` now includes `make docs-pdf-check`, which fails when the PDF no longer matches its sources. See build.

2026-10-13: clearer column headers on the Results tab

Two per-track columns on the *Results* tab have new headers:

- *Purity* is now *Most Frequent* and names the vehicle the share belongs to, e.g. `75 % (obu1)`; on a tie it names each of them.
- *Stations, in order* is now *Station predictions in actual order across identity changes*.

The values are the same, and `score.json` keeps its fields (`purity`, `main_station`, `stations`). The track table of the summary report (`report.docx`, section 3) has the same two columns; it had no *Most Frequent* column before. Reports already written are unchanged until they are made again (`python3 -m vnapapi.report <results directory>`).

2026-10-12: run identifiers from the scenario name

A run's identifier is `<user>-<scenario name>-<n>` (e.g. `alice-mixzone-random-3`) instead of 12 hex digits. `n` starts at 1 and counts that user's runs started under that name; the database keeps the counters (new table `run_counters`, made on the first start), so a number is never given twice. The name is the one the run is started with; user and name are reduced to letters, digits, `.`, `_` and `-`. Existing runs keep their identifiers.

- A scenario is validated before the run gets its number, so a refused one uses none (and no longer leaves its text in `data/scenarios`).
- The broker account's environment variables no longer carry the run id.
- Fixed on the way: a station `name` that is not text (`name = 1`) is reported as a scenario error instead of failing the request.

2026-10-12: suggested next runs in the summary report

The summary report ends with "6. Suggested next runs", chosen by rule from the run's measurements. The rules are in `service/recommendations.toml`: fixed text with conditions on the run's facts (`links.wrong>=8`, `pattern.share>=0.8`), priorities and a limit of 6. The 16 starting rules come from the recommendations of the summaries in `results/`.

- **Editable without code:** the file is read for every report. `python3 -m vnapapi.report --rules <results directory>` lists a run's facts and what each rule made of them, including rules with a mistake.

- **No judgement:** a rule only says what was written into it; each suggestion states the number that triggered it.
- Details: suggested next runs.

2026-10-12: summary report at the end of a run

When a run stops, the service writes `report.docx` into its results, in the form of the test summaries in `results/`: header table, purpose, setup, results (with every link scored against ground truth, wrong links by vehicle pair and the content of each track), observations, conclusion and limitations. The *Results* tab links it first.

- **Facts only** in this first version; suggested next runs were added the same day (above).
- **Built from the collected files** (`service/vnapapi/report.py`, standard library only), so `python3 -m vnapapi.report <results directory>` writes it again, also for earlier runs.
- `run.json` gains `stopped_at`, `stop_reason` and the last check; `score.json` gains each link's `evidence`.
- Details: summary report.

2026-10-11: run name and description

A run has a name and a description of its own, which start as its scenario's. The owner (or an admin) changes them on the web UI's Overview tab, or with `PATCH /api/runs/{id}`, in any state of the run. The runs list shows the name; `run.json` in the results records both. The database gains the columns on the first start (runs from before show their scenario's name and description).

2026-10-11: earlier events on the Events tab

The Events tab shows the events of the last 10 minutes when it opens (it showed only those of the last 5 s, so it stayed empty until the next event), and says when it is connected with nothing to show. `GET /api/runs/{id}/events/stream` takes `since_s` and sends an event `ready` after the earlier events.

2026-10-11: vehicle colours

A station's `color = "#rrggbb"` sets the colour of the station and its route on the web UI's maps (the builder and the run's Map tab). The builder has a colour picker per vehicle.

2026-10-11: dynamic mix zones

The mobility client sends a pseudonym change event to a vehicle when at least `min_vehicles` participating vehicles are within `radius_m` of it, at most every `cooldown_s`: `[control.dynamic_mix_zone]` for the run, `dynamic_mix_zone = { enabled, min_vehicles }` per station. Scenario `c-its-pki-convoy-dynamic`; the web UI's builder has "all vehicles" and "min cars" for the run and a toggle with "min cars" per vehicle. No new station image.

2026-10-11: new run from a template, saved templates

- **New run:** the page starts from a template, whose text becomes the new scenario. The "Scenario text" and "Builder" tabs edit that same scenario: the builder reads the text (`POST /api/scenarios/parse`) and keeps the keys its forms do not show. An unchanged template is still started by its name.
- **Saved templates:** a scenario can be saved as a template of one's own, optionally shared with all users (`/api/templates`; a new database table).
- **Generic names:** saving can rename stations to `rsu1…`, `vehicle1…` and mix zones to `zone1…`.

- `POST /api/runs` takes a `name` for the run.

2026-10-11: line breaks in scenario text

A line break inside a `"..."` string of a scenario text is refused by TOML ("Illegal character '\n'"). The error now says how to write it (`"""..."""`), and the web UI's text editor says so above the text.

2026-10-10: mix zone threshold

`[[control.mix_zones]]` takes `min_vehicles = n`: a vehicle that entered the zone changes its pseudonym only once at least `n` watched vehicles are inside. Those waiting change together; one that leaves before keeps its pseudonym. Scenario `c-its-pki-mixzone-min3`; the web UI's builder has the field per mix zone. The web UI's Control tab no longer has the "Change pseudonym" button, which did the same as "ID change trigger".

2026-10-10: `cookie_secure = false` in the repository's configuration

`service/config.toml` sets `cookie_secure = false`, for testing only: login then works over plain HTTP in every browser, including Safari and DuckDuckGo. A production deployment sets `true` (a step of the deployment procedure); the service's built-in default stays `true`.

2026-10-10: installation paths in the documentation

The documentation names the clone's parent directory `INSTALL_DIR` and the repository `VNAP_HOME` (`${INSTALL_DIR}/vnap-secure`) instead of `~/COIMBRA`; commands change directory with `cd ${VNAP_HOME}/…`. The scripts do not read these variables.

2026-10-10: vnapctl progress messages

`vnapctl` reports what it is doing on stderr (`vnapctl: …`): building a helper image, starting the run's PKI and the stations, waiting for readiness, collecting events. `-q`/`--quiet` and `--json` turn the messages off. `up` pulls `eclipse-mosquitto:2` before the readiness wait instead of during it.

2026-10-09: OpenAPI description in the repository

`docs/reference/openapi.json`: the API's OpenAPI 3.1 description, as the service serves it at `/api/openapi.json`. `make openapi` regenerates it (`python3 -m vnapapi.openapi`), and `make test` checks that it is current.

2026-10-09: repository reorganization

Implements `Specs/VNAP-Secure-Repository-Reorganization-Plan.docx` (decisions D1–D9).

- **Layout:**
  - `vnap-patches/` and `vnap-origs/` became `patches/vanetza-nap/modified/` and `upstream/`;
  - `vnap-certs/` became `certs/`;
  - `vnap-docker/` became `sim/`, with the helper images in `sim/images/`;
  - the service moved from `vnap-docker/api/` to `service/`;
  - `TestSummaries/` became `results/`;
  - the top-level scripts moved to `scripts/{build,vm,certs}/`;

  - `r2-entrypoint.sh` became `sim/stock-entrypoint.sh`.
- **Removed:**
  - the docker-compose harness (`docker-compose.yml`, `docker.env`, `start-vnap.sh`, `stop-vnap.sh`);
  - the `run-r2-sim.sh` harness and its helpers (`end-r2-sim.sh`, `check-r2-cams.sh`, `rsu-log.sh`, `obu-log.sh`), all superseded by `vnapctl`;
  - the duplicate `vnap-certs/gen-certify.sh`;
  - the expired certify AT `ticket_vnap`.

  Everything removed remains at the tag `pre-reorg-2026-10`.
- **C-ITS-PKI** is a git submodule (`external/C-ITS-PKI`, pinned). `CITS_PKI_DIR` still overrides it, and a checkout next to vnap-secure is used for one transition release. The per-run PKI image, the PKI container and `run.json` record the C-ITS-PKI commit (`vnap.cits_pki`), and `vnapctl status` shows it.
- **vanetza-nap** stays outside the repository: `patches/vanetza-nap/BASE` records the upstream commit, and `make image` clones it when `VANETZA_NAP_DIR` (default `~/vanetza-nap`) is missing.
- **New:**
  - a `Makefile` (image, msgcheck, test, diffs, docs-check);
  - generated unified diffs of the patch set;
  - a documentation link checker;
  - `docs/` with installation, build, operations, reference, functional and patch documentation.
- **Paths** are no longer hard-coded in the scripts: `VM_IP`, `VM_USER`, `SSH_KEY`, `VANETZA_NAP_DIR`, `OUTPUT_DIR` and `VNAP_SIM_DIR` select them.

2026-10-08

- **Hardening (PRD phase 9, without the MCP endpoint):**
  - an optional TOTP second factor with recovery codes; the secrets are encrypted at rest;
  - backups of the database, results and user scenarios, with verify, restore and a daily timer.
- **Web UI (phase 8):**
  - the runs list, and new runs from templates, scenario text or a map-based builder;
  - a run view with overview, live map, events, control and results;
  - account and admin pages.

  Later fixes: plain-HTTP login detection, the map tile Referer, clearer certificate-check events, the scenario description on the overview, and the eavesdropper's "longest chain followed" score with explanations.
- **API service (phase 7):**
  - accounts, roles, sessions with CSRF protection, API tokens, lockout, audit log;
  - runs as jobs with per-role limits;
  - live events, control actions, checks and results.
- **Coimbra origin:** the example layouts moved from Vanetza-NAP's built-in position (40, −8) to the crossing of Rua Alexandre Herculano and Rua Venâncio Rodrigues, Coimbra; stations without a position default to it.

2026-10-07

- `vnapsim` **(phase 6):**
  - `vnapctl` became a launcher for the `vnapsim` package;
  - the JSON Schema of the scenario format and field-level validation errors;

  - service-assigned identities, the user policy, and atomic instance allocation (`--instance auto`);
  - content-addressed helper images.
- **Station key derivation (phase 5):** the station derives its butterfly ATs' private keys; the PKI holds none (`vnap:r2-p19`).
- **Refill measurements (phase 4):** batch-size sweep and mix zone with the run's own PKI (`results/pki-refill-test-2026-10-07.docx`).
- **Per-run PKI and certificate refill (phases 1–3):** the `vnap-pki` service and the growing pseudonym pool (`vnap:r2-p18`), driven by `vnapctl`.

2026-10-03 – 10-04

- Vehicle movement: the position control channel, the mobility client, traffic and convoy scenarios (`vnap:r2-p16`).
- Intersection mix zones, with fixed and random turns; test summaries in `results/`.
- The CAM timer rephase on ID change (`vnap:r2-p17`), and timing-phase linking in the eavesdropper.
- The ETSI ID change notification service (TS 102 723-8/-9): GN address, MAC and `stationId` change with the certificate; silent periods after ID changes.

2026-09-28 – 10-01

- The release2 patch set, replacing the earlier patch set for Vanetza-NAP `main`.
- Pseudonym pool and event-driven pseudonym changes on a separate control channel.
- `vnapctl`: status, events and checks, then up/down with TOML scenario files and parallel instances.
- Thread-safety patches, all found with ThreadSanitizer: sign header policy, v2/v3 certificate caches, runtime clock and ASN.1 descriptors, router threads, PubSub/MQTT/DDS, RSSI reader.
- The passive eavesdropper (tracking attacker).
- `vnap-msgcheck`: verify message files with Vanetza's own security code.

Before 2026-09

Until 2026-09-28 this repository held a patch set for Vanetza-NAP `main`:

- parser fixes;
- an LRU-cache mutex;
- debug and metrics logging;
- pseudonym rotation (`pseudonym_certificate_provider`);
- a docker-compose-free harness;
- a kind/K8s attempt.

It was replaced by the release2 patch set on 2026-09-28. The pseudonym rotation was ported to release2 the same day, and its fixed countdown was later replaced by change events on a control channel. The committed versions of that work remain in git history (from the first commit, 2026-05-21, to `6beadfb`). Files that were never committed are archived in `~/vnap-secure-removed-dirs-20260928.tar.gz` in the development VM.

PRD phases

`Specs/VNAP-Secure-Simulation-Service-PRD.pdf` plans the service in nine phases:

Phase	Content	Done
1–2	PKI service mode; station refill (option A)	2026-10-07
3	`vnapctl`: certificate policy, per-run PKI	2026-10-07
4	Measurements	2026-10-07
5	Station key derivation (option B)	2026-10-07
6	`vnapctl` library	2026-10-07
7	API service	2026-10-08
8	Web UI	2026-10-08
9	Hardening: backups, TOTP	2026-10-08; MCP endpoint not yet

Documentation moves

The README used to hold all documentation. Its sections are now here:

Former README section	Now
Contents	README
Getting started	README, vnapctl
Development setup	installation
Dependency on C-ITS-PKI	architecture
Build	build
Run the simulation: options 1 and 2	removed (harnesses retired)
Option 3: vnapctl with scenario files; vnapctl (prototype)	vnapctl
The vnapsim package, validation and the user policy	scenario format, architecture
Container environment	container environment
Simulation service (API)	service deployment, API
Web UI	web UI
Hardening (phase 9)	backup and restore, service deployment
Monitor messages, application messages (MQTT), tcpdump	monitoring
Pseudonym control channel	control channels
Pseudonym change events	pseudonym change
Certificate refill (per-run PKI)	certificate refill
Vehicle movement	vehicle movement
Eavesdropper (tracking attacker)	eavesdropper and scoring
Certificates	certificates
Patch set	patch documentation
Validation and troubleshooting	troubleshooting
History	this file

ASN.1 modules

sim/images/eavesdropper/asn1/README.md

Copied unchanged from Vanetza-NAP (asn1/, asn1/release2/), which bundles them from the standardisation documents. They are **not** under the license of this repository; see the referenced documents for the applicable IPR. Vanetza notes that ETSI does not object to bundling the ASN.1 of its published documents with software.

File	Source
IEEE1609dot2BaseTypes.asn, IEEE1609dot2.asn	IEEE 1609.2, as printed in ETSI TS 103 097 V1.3.1
TS103097v131.asn	ETSI TS 103 097 V1.3.1 (2017-10)
TS102894-2v221-CDD.asn	ETSI TS 102 894-2 V2.2.1 (Common Data Dictionary)
TS103900v211-CAM.asn	ETSI TS 103 900 V2.1.1 (CAM, release 2)